

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Jakub Rollo

Improving Super Mario level generation with perfect agents

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: Mgr. David Šosvald
Study programme: Computer Science

Prague 2025

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedicated to Julinka and Maggie for their unwavering support.

Title: Improving Super Mario level generation with perfect agents

Author: Jakub Rollo

Department: Department of Software and Computer Science Education

Supervisor: Mgr. David Šosvald, Department of Software and Computer Science Education

Abstract: In modern video games, procedural content generation is widely used to automatically create game content, with one of its key applications being level generation. It is essential to ensure that these procedurally generated levels are playable, and this thesis focuses on enhancing the validation process specifically in the context of Super Mario Bros. The main objective is to improve the existing evaluation method, which utilizes artificial agents, by replacing the commonly used A*-based agent developed by Robin Baumgarten with more capable agents from David Šosvald.

The thesis builds on three existing studies on Super Mario Bros. level generation, all of which used the Baumgarten's agent for playability evaluation, with two of them also employing it during the generation process itself. These studies are examined in detail, including their methodologies. Their results were both replicated and extended using the improved agents. A comparative analysis was conducted between the original and improved agents based on level completion rates, generation speed, fitness convergence, and the extent to which generation goals were achieved.

The results demonstrate that the new agents completed a higher number of generated levels, reached better fitness values in fewer generations, and produced levels that more consistently met the specified criteria, all while significantly reducing generation time. To support future research, comprehensive documentation was created for the source code of the studies, addressing the lack of detailed documentation in the original works.

Keywords: procedural generation, Super Mario, artificial intelligence, computer game, level validation

Název práce: Vylepšení generování Super Mario úrovní pomocí dokonalých agentů

Autor: Jakub Rollo

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: Mgr. David Šosvald, Katedra softwaru a výuky informatiky

Abstrakt: V moderních videohrách se k automatické tvorbě herního obsahu často využívá procedurální generování, přičemž jedním z jeho klíčových využití je generování úrovní. Je nezbytné zajistit, aby takto generované úrovně byly skutečně dokončitelné, proto se tato práce zaměřuje na vylepšení jejich validace, konkrétně v kontextu hry Super Mario Bros. Hlavním cílem je zdokonalit stávající metodu vyhodnocování, která využívá umělé agenty, a to nahrazením běžně používaného agenta na bázi A*, vyvinutého Robinem Baumgartenem, výkonnějšími agenty od Davida Šosvalda.

Tato práce navazuje na tři existující studie zabývající se generováním úrovní ve hře Super Mario Bros, z nichž všechny používaly Baumgartenova agenta pro hodnocení dokončitelnosti, přičemž dvě z nich ho využívaly i při samotném procesu generování. Tyto studie jsou podrobně rozebrány, včetně popisu jejich metodik. Jejich výsledky byly replikovány a rozšířeny použitím lepších agentů. Dále bylo provedeno srovnání mezi Baumgartenovým agentem a výkonnějšími agenty na základě úspěšnosti dokončení úrovní, rychlosti generování, konvergence fitness hodnot a schopnosti dosahovat cílů generování.

Výsledky ukazují, že noví agenti dokončili větší počet generovaných úrovní, dosáhli lepších fitness hodnot v menším počtu generací a vytvořili úrovně, které lépe splňovaly zadaná kritéria, a to vše při výrazném zkrácení doby generování. Pro podporu budoucího výzkumu byla vytvořena souhrnná dokumentace zdrojových kódů použitých studií, která řeší nedostatek podrobné dokumentace v původních pracích.

Klíčová slova: procedurální generování, Super Mario, umělá inteligence, počítačová hra, validace levelů

Contents

1 Introduction **8**
- 1.1 Goals . . . 9
- 1.2 Structure of the Thesis . . . 9
- 1.3 Copyright . . . 10
- 1.4 Acknowledgments . . . 10

2 Theory **11**
- 2.1 Super Mario Bros.: A Platform for Research . . . 11
  - 2.1.1 Introduction to Super Mario Bros. . . . 11
  - 2.1.2 Super Mario Bros. as a Research Platform . . . 12
  - 2.1.3 The Mario AI Framework . . . 12
- 2.2 Procedural Content Generation (PCG) . . . 13
- 2.3 Key Generation Techniques . . . 13
  - 2.3.1 Generative Adversarial Networks (GANs) . . . 13
  - 2.3.2 Evolutionary Algorithms . . . 15
- 2.4 Level Validation: Ensuring Playability . . . 16
- 2.5 AI Agents for Automated Playtesting . . . 16
  - 2.5.1 The A* Search Algorithm . . . 17
  - 2.5.2 Applying A* to Super Mario Bros. . . . 17

3 Related Work and Study Selection **21**
- 3.1 Selection of Studies for Re-evaluation . . . 21
  - 3.1.1 DagstuhlGAN . . . 22
  - 3.1.2 TOAD-GAN . . . 22
  - 3.1.3 MechanicsGen . . . 22
- 3.2 Summary of Scope . . . 23

4 TOAD-GAN **24**
- 4.1 TOAD-GAN Methodology . . . 24
  - 4.1.1 Core Architecture: Adaptation of SinGAN . . . 24
  - 4.1.2 Token Representation and Downsampling . . . 25
- 4.2 Original Evaluation and Shortcomings . . . 26
- 4.3 Replication and Improved Evaluation . . . 26
- 4.4 Results and Analysis . . . 27
- 4.5 Chapter Summary . . . 29

5 DagstuhlGAN **30**
- 5.1 DagstuhlGAN Methodology . . . 30
  - 5.1.1 GAN Architecture and Training . . . 30
  - 5.1.2 Latent Variable Evolution (LVE) with CMA-ES . . . 31
  - 5.1.3 Original Fitness Functions . . . 32
- 5.2 Original Evaluation and Shortcomings . . . 32
- 5.3 Replication and Extension . . . 33
  - 5.3.1 Replication Challenges and Setup . . . 33
  - 5.3.2 Framework Update and Agent Replacement . . . 34

5.3.3 New Fitness Functions . . . 34
5.4 Results and Analysis . . . 36
5.4.1 Agent Comparison with Original Jump Fitness . . . 36
5.4.2 Agent Comparison with New Fitness Functions . . . 39
5.4.3 Overall Analysis . . . 44
5.5 Chapter Summary . . . 44

6 MechanicsGen **45**
6.1 MechanicsGen Methodology . . . 45
6.1.1 Core Approaches . . . 45
6.1.2 Scene Representation and Evaluation . . . 46
6.2 Original Evaluation and Shortcomings . . . 47
6.3 Replication and Extension . . . 47
6.3.1 Code Completion and Bug Fixing . . . 47
6.3.2 Agent Integration and Optimization . . . 48
6.3.3 Addressing Agent Non-Determinism . . . 48
6.3.4 Multithreading and Caching . . . 49
6.4 Results and Analysis . . . 50
6.4.1 Agent Performance Comparison . . . 50
6.4.2 Improved Agent Configuration . . . 50
6.4.3 Combined Optimizations and Fitness Evolution . . . 53
6.5 Chapter Summary . . . 54

7 Additional Scripts and Documentation **55**
7.1 TOAD-GAN . . . 55
7.2 DagstuhlGAN . . . 55
7.3 MechanicsGen . . . 55
7.4 Helper Python Scripts . . . 56
7.5 Metacentrum Job Scripts . . . 56

8 Conclusion and Future Work **57**
8.1 Summary of Key Findings . . . 57
8.2 Contributions . . . 58
8.3 Limitations and Future Work . . . 58

Bibliography **60**

List of Figures **63**

List of Tables **66**

List of Abbreviations **67**

A Attachments **69**
TOAD-GAN Documentation . . . 69
DagstuhlGAN Documentation . . . 71
MechanicsGen Documentation . . . 75
Helper Python Scripts . . . 78
Metacentrum Scripts . . . 79

1 Introduction

Procedural Content Generation (PCG) refers to the automatic creation of content using computational techniques such as rule-based systems, stochastic methods, or machine learning. It has been applied in various domains, including architecture [1], art [2], and entertainment. In the context of digital games, PCG is often used to reduce manual work in content creation and to increase variety and replayability. One of its most prominent applications in games is level generation, where a key challenge lies in ensuring that the generated levels are both playable and engaging [3].

The *Super Mario Bros.* game, released by *Nintendo* in 1985 [4], has become a common benchmark for research in PCG and artificial intelligence due to its popularity, simplicity, and well-defined mechanics. Several frameworks and studies have been created over the years that focus on generating Super Mario Bros. levels using procedural generation. A key component of such systems is the evaluation of playability, which is typically assessed by using intelligent agents to simulate level completion. Historically, one of the most widely used agents for this task has been *Robin Baumgarten's A* agent*, which won the 2009 Mario AI Competition [5] and became the accepted standard in the field.

Despite being a standard choice, Baumgarten's agent suffers from critical limitations: it is inefficient and often fails on levels that are, in fact, completable [6]. These weaknesses are especially problematic because many procedural level generation studies rely on this agent, either during the generation process itself or for post-generation evaluation. For this reason, the agent not only misclassifies completable levels as unplayable, but can also inadvertently steer the direction of generation algorithms during training or evolution. When this happens, generators risk discarding promising level designs or overfitting to levels aligned with the limitations of the agent rather than actual playability. As a result, the findings of these studies might be held back by flaws in the evaluation method they rely on.

This thesis builds upon three key studies that employed Baumgarten's agent, selected based on the criteria discussed in Chapter 3. The goal is to revisit and enhance level evaluation in these studies:

- *TOAD-GAN: Coherent Style Level Generation from a Single Example* (referred to as *TOAD-GAN*) [7]
- *Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network* (referred to as *DagstuhlGAN*) [8]
- *Intentional Computational Level Design* (referred to as *MechanicsGen*) [9]

To address the limitations of Baumgarten's agent, this thesis replaces it with two existing, significantly enhanced alternatives: the *MFF A** agent [6] and the *MFF A* Grid* agent [10]. Compared to Baumgarten's original implementation, the MFF agents offer greater reliability, faster execution, and higher completion rates across levels [6]. By replicating the results and re-evaluating the three previously mentioned studies using this improved agent while comparing key outcomes, we aim to investigate the extent to which previous research may have been affected by relying on a suboptimal agent.

1.1 Goals

The primary goal of this thesis is to improve the evaluation phase of procedural level generation by integrating the MFF agents into existing PCG systems. Specifically, we aim to:

- Reproduce the experiments presented in the TOAD-GAN, DagstuhlGAN, and MechanicsGen papers.
- Replace Baumgarten's agent with the MFF agents in all evaluation processes.
- Compare the performance of both agents in level completability, convergence of fitness functions, and generation speed.
- Analyze the completability and quality of the generated levels by both agents.
- Address missing documentation in the replicated and extended systems where necessary.

1.2 Structure of the Thesis

The thesis will follow this structure:

- **Chapter 2: Theory** — Introduces the theoretical background relevant to this thesis: generative adversarial networks, evolutionary algorithms, and artificial agents.
- **Chapter 3: Related Work and Study Selection** — Reviews Super Mario level generation papers using Baumgarten's agent and explains the selection of the three studies.
- **Chapter 4: TOAD-GAN** — Summarizes and replicates the TOAD-GAN paper, replacing Baumgarten's agent with the MFF A* Grid agent for evaluation. Compares level completion results between the two agents.
- **Chapter 5: DagstuhlGAN** — Summarizes and replicates the DagstuhlGAN study, extends it with MFF agents, introduces custom fitness functions and analyzes changes in level evolution and level quality.
- **Chapter 6: MechanicsGen** — Summarizes and replicates the MechanicsGen paper, integrates MFF A*, improves the implementation, and analyzes its impact on the generated levels and the generation process.
- **Chapter 7: Additional Scripts and Documentation** — Describes the supporting scripts, tools, and technical documentation.
- **Chapter 8: Conclusion and Future Work** — Summarizes the key findings of the thesis and outlines possible directions for future research.

1.3 Copyright

All rights to Super Mario Bros. and its assets belong to Nintendo. This thesis is an academic work and makes use of existing research frameworks and level data on a non-commercial basis for research and educational purposes only.

1.4 Acknowledgments

Computational resources were provided by the *e-INFRA CZ* project (ID:90254) on the MetaCentrum grid, supported by the *Ministry of Education, Youth and Sports of the Czech Republic*. Additional resources were provided by the *ELIXIR-CZ project* (ID:90255), part of the international ELIXIR infrastructure [1].

The thesis was created using the official LaTeX template for student theses provided by the Faculty of Mathematics and Physics, Charles University, created by Mgr. Martin Mareš, Ph.D. [2]

ChatGPT by OpenAI was used to improve the clarity and style of selected parts of the text.

[1] `https://metavo.metacentrum.cz/`

[2] `https://www.mff.cuni.cz/cs/studenti/sablony-studentskych-praci`

2 Theory

This chapter lays the theoretical groundwork for the research presented. It begins by introducing Super Mario Bros. as a platform for AI and PCG studies. Following this, an overview of PCG principles is provided, leading into a detailed examination of key generative techniques employed in this thesis, specifically Generative Adversarial Networks (GANs) and Evolutionary Algorithms (EAs). The chapter then addresses the critical aspect of level validation, emphasizing methods to ensure completability. Finally, it delves into the AI agents utilized for automated playtesting of Super Mario levels and their validation, with a particular focus on the A* search algorithm, discussing both the historically significant Baumgarten's A* agent for Super Mario Bros. and subsequent advanced adaptations, including the Slim Forward Model and the MFF agents. These advanced agents, specifically the MFF A* agent and the MFF A* Grid (BND) variant serve as the proposed superior alternatives for level validation in this thesis.

2.1 Super Mario Bros.: A Platform for Research

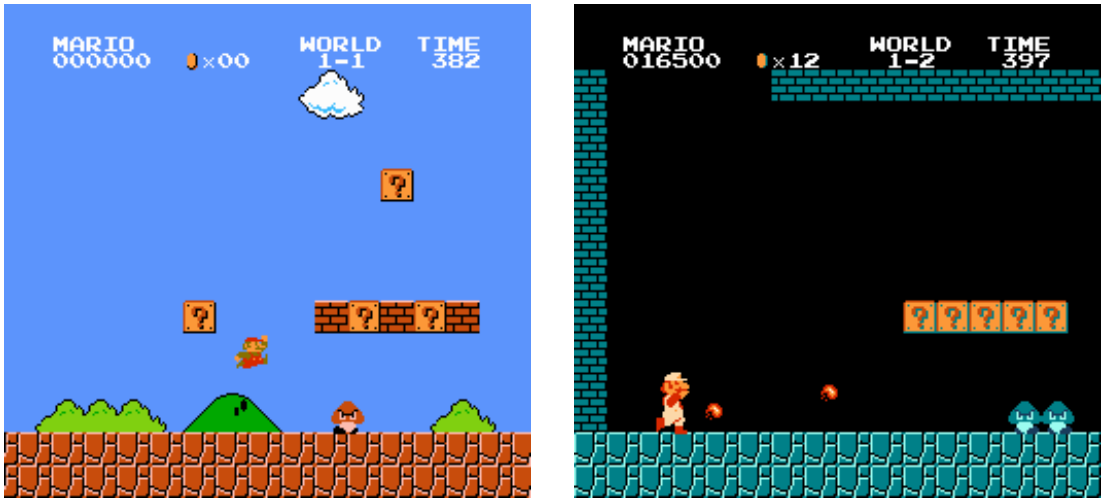
2.1.1 Introduction to Super Mario Bros.

Super Mario Bros., released by Nintendo in 1985 for the Nintendo Entertainment System (NES) [4], is an influential title in video game history and remains an iconic platformer. The player controls Mario, navigating him through two-dimensional, side-scrolling levels.

The game's mechanics are simple to learn but together can offer surprisingly complex interactions. Core actions are walking and running (with a dedicated button enabling higher speeds), jumping (where holding the jump button makes Mario jump higher), and interacting with the environment and enemies. You can jump on enemies such as Goombas and Koopa Troopas to defeat them, hit blocks from below, and collect items such as coins and mushrooms.

Each game level consists of different types of tile types such as solid ground blocks, breakable brick blocks, question mark blocks (containing power-ups or coins), and pipes that can serve as obstacles or transport Mario to different areas. Power-ups, such as the Super Mushroom (transforming Mario into Super Mario, allowing him to break bricks and sustain one extra hit), and the Fire Flower (allowing Super Mario to shoot fireballs), add another layer of complexity to the gameplay.

Levels often contain gaps that require precise jumps, along with enemy placements that create additional challenges. The game features a variety of level types, including overworld, underground, castle, and mushroom environments, each presenting distinct challenges.



(a) Screenshot from World 1-1 **(b)** Screenshot from World 1-2

Figure 2.1 Gameplay screenshots taken from the levels World 1-1 and World 1-2 in Super Mario Bros. [11, 12].

2.1.2 Super Mario Bros. as a Research Platform

The discrete, tile-based nature of Super Mario Bros. levels, combined with its well-defined, simple physics, rich design patterns, and clear objectives, makes it a suitable platform for research in artificial intelligence (AI) and PCG. Its popularity ensures familiarity, while the established mechanics provide a constrained yet interesting environment for testing algorithms in level design [8, 7], agent control [9], and automated playtesting [6]. Numerous frameworks and tools, such as the Mario AI Framework[13], have been developed, further solidifying its role as a standard benchmark in the field.

Levels are typically represented as a 2D grid, where each cell contains a specific tile type, making them easy to manipulate and analyze [14].

2.1.3 The Mario AI Framework

The *Mario AI Framework* is a key software tool facilitating much of this research. Originally based on Infinite Mario Bros. by Markus Persson, an open-source reinterpretation inspired by the original game [13], the framework provides the core game engine, standardized level representations (human-readable text-based grids, where different symbols correspond to various tile types), and interfaces for developing and testing both level generators and AI agents. It has hosted several AI competitions [5, 15], driving innovation in agent design and PCG techniques.

The framework used in this thesis is an evolution of these earlier versions, notably incorporating the MFF A* agent, the MFF A* Grid agent and the improved *Slim Forward Model* for the agents [6, 10, 16].

2.2 Procedural Content Generation (PCG)

Procedural Content Generation (PCG) refers to the algorithmic creation of content, such as levels, items, quests, textures, or even narratives, through procedures or algorithms, rather than the manual creation by designers. Though PCG is most commonly associated with video games, its methodology and techniques have been applied across a variety of different fields, including art [2], architecture [1], design, and scientific research. In the context of video games, PCG is often used to create dynamic and varied content that can provide new experiences with every playthrough [3].

A prominent application of PCG is the generation of levels. Here, algorithms are responsible for designing the playable spaces within a game. However, generating high-quality levels automatically presents several significant challenges. The first among these is verifying that a generated level can actually be completed by a player (or an AI agent representing one). Beyond only ensuring completability, level generators should also address several other crucial aspects. These include controlling the difficulty curve [8], maintaining aesthetic and structural coherence consistent with the game or previous levels [17], and generating meaningful or interesting challenges [9, 18]. The difficulty in guaranteeing these properties, especially playability, requires robust evaluation and validation methods within the PCG pipeline [19, 9].

2.3 Key Generation Techniques

The studies central to this thesis utilize several advanced computational techniques for level generation, mainly drawing from machine learning and evolutionary computation.

2.3.1 Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) [20] are a class of deep learning models known for their ability to generate synthetic data that mimics a given training dataset. A GAN consists of two neural networks trained simultaneously in a competitive setting:

- **Generator (G):** Takes random noise (typically a vector sampled from a latent distribution, such as a normal or uniform distribution) as input; this vector, known as a latent vector, represents the abstract features in a compressed form. The generator then attempts to map this noise into data (e.g., an image, a level segment) that resembles the real training data.
- **Discriminator (D):** Takes both real data samples from the training set and fake data samples generated by G as input. It attempts to classify whether its input is real or fake.

During training, G learns to produce increasingly realistic data to fool D, while D learns to become better at distinguishing real from fake. This adversarial process drives G to learn the underlying distribution of the training data [20].

In the context of level generation, GANs can be trained on datasets of existing game levels or level segments. Once trained, the generator can produce new levels that capture the style and patterns present in the training data. Several architectures relevant to this thesis are discussed below, including *Deep Convolutional GANs* (DCGANs), *SinGAN*, and *Wasserstein GANs* (WGANs).

Deep Convolutional GANs (DCGANs)

Introduced by Radford et al. [21], DCGANs use deep convolutional neural networks for both the generator and the discriminator, largely replacing fully connected layers. A key feature of this architecture is the use of *convolutional layers* in the discriminator to gradually *downscale* spatial data and extract hierarchical features - these layers apply filters that reduce the resolution (e.g., from 32×32 to 16×16) while increasing the number of feature maps (also known as channels), enabling the network to learn spatial patterns effectively. The generator, by contrast, employs *transposed convolutional layers* (also known as *fractionally strided convolutions*) to *upscale* the latent vector back into a full-sized image or level. These layers increase spatial resolution (e.g., from 8×8 to 16×16) and are needed for creating coherent, high-resolution output from low-dimensional noise. Batch normalization is typically applied throughout to stabilize and accelerate training. Due to their grid-based processing, DCGANs are particularly well-suited for image-like data such as game levels. For instance, the DagstuhlGAN study [8] employs a DCGAN architecture, utilizing a Wasserstein GAN (WGAN) loss function (discussed further below in the subsection on WGANs) for improved training stability. The individual GANs within more complex architectures, such as SinGAN (also detailed below), typically follow DCGAN principles regarding their convolutional structure.

SinGAN

Proposed by Shaham et al. [22], SinGAN is notable for its ability to learn a generative model from a single training image (or level). It achieves this through a hierarchical structure, essentially a pyramid of GANs trained at different spatial scales of the input. Generation starts at the coarsest scale, and the output is progressively upsampled and refined by GANs at finer scales. This allows it to capture both global structure and fine details, making it suitable for generating coherent content based on one example. To ensure stable training at each level of the pyramid, SinGAN uses the *WGAN-GP* loss function [23] (detailed in the following subsection on WGANs). The TOAD-GAN study [7] specifically adapts the SinGAN architecture for token-based level generation.

Wasserstein GANs (WGANs)

Wasserstein GANs (WGANs) [24] aim to improve the stability of GAN training and reduce issues like mode collapse, where the generator produces only a limited variety of outputs. They do this by using a different loss function based on the Wasserstein distance, which provides a more meaningful measure of how close the generated data is to the real data. This helps the generator learn more effectively and makes training more stable. A later improvement, WGAN-GP (WGAN with

Gradient Penalty) [23], replaces an earlier method that limited how much the model's values could change (called weight clipping) with a smoother technique that gently penalizes large changes (called a gradient penalty). This results in even better training stability and sample quality. As mentioned, DagstuhlGAN [8] uses WGAN techniques within a DCGAN framework. Similarly, the SinGAN architecture [22], and TOAD-GAN [7] which builds on it, use WGAN-GP to train their hierarchy of generators.

2.3.2 Evolutionary Algorithms

Evolutionary Algorithms (EAs) are a family of population-based metaheuristic optimization algorithms inspired by biological evolution. They are well suited for complex search problems where traditional optimization methods may struggle. [25] The core concepts include:

- **Population:** A set of candidate solutions (individuals) to the problem. In PCG of levels, an individual might represent a complete level or the parameters controlling a generator [26, 25].
- **Fitness Function:** A function that evaluates the quality or suitability of each individual in the population according to the desired objectives (e.g., playability, specific features, aesthetic metrics) [27, 25].
- **Selection:** A process where individuals with higher fitness are more likely to be chosen to contribute to the next generation [25].
- **Variation Operators:** Mechanisms such as mutation (small random changes to an individual) and crossover (combining parts of two parent individuals) are used to create new offspring, exploring the search space [25].

Through iterative cycles of evaluation, selection, and variation, EAs gradually guide the population toward regions of the search space containing high-quality solutions. Specific EAs are employed in the target studies:

Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is an evolutionary algorithm used for solving optimization problems with continuous values. It is often used when the structure of the search space is not well understood or is difficult to model with gradients. CMA-ES works by trying out new candidate solutions and adjusting how it searches based on which solutions perform well. Over time, it learns which directions in the search space are more promising. In the DagstuhlGAN study [8], CMA-ES is used to explore the latent space of a pre-trained GAN generator, aiming to find inputs that produce level segments with specific desired properties, such as better playability or more jumps. [27]

Feasible-Infeasible 2-Population (FI2Pop)

Feasible-Infeasible 2-Population (FI2Pop) is an evolutionary algorithm specifically designed to handle constrained optimization problems, where solutions must satisfy certain requirements (constraints) in addition to optimizing an objective

function. FI2Pop maintains two co-evolving populations: one containing *feasible* individuals (those that satisfy all constraints) and one containing *infeasible* individuals (those that violate at least one constraint). Feasible individuals are optimized based on the primary objective function, while infeasible individuals are optimized based on minimizing their constraint violation. This separation allows the algorithm to effectively explore the boundary between feasible and infeasible regions, which is often where optimal constrained solutions lie. [26] The MechanicsGen study utilizes FI2Pop to evolve levels that meet specific gameplay constraints, such as being completable by a 'perfect' agent but not by a 'limited' agent (e.g., one unable to perform high jumps), thereby enforcing the use of a particular mechanic.

MechanicsGen also explores another EA variant called *Constrained MAP-Elites* (CMElites) [28]. It maintains a diverse archive of solutions across a feature space (e.g., used mechanics) while filtering for feasibility, allowing it to generate a range of levels that satisfy specific goals.

2.4 Level Validation: Ensuring Playability

A critical step in any PCG of levels pipeline is validation. Levels generated algorithmically are not inherently guaranteed to be functional or to meet design goals [8, 7]. Ensuring playability – that a level can be successfully navigated from start to finish – is arguably the most fundamental validation requirement. Without it, generated levels are unusable.

Two main approaches exist for level validation. *Static analysis* involves examining the level structure directly (e.g., checking for unreachable platforms, blocked passages, excessively large gaps) without simulating gameplay [29, 30]. This can be fast but often fails to capture the dynamic complexities of gameplay (e.g., enemy behavior, physics-based movement). *Simulation-based validation*, conversely, employs an AI agent to attempt playing through the generated level. This provides a more accurate assessment of dynamic playability, as it directly tests whether the required actions and interactions are possible within the game's rules and physics. While more computationally expensive, simulation-based validation using AI agents is the primary method used for ensuring playability in the studies central to this thesis [7, 8, 9, 6].

To understand the specific validation methods employed in these works, we must first examine the types of agents they use. Most notably, these agents rely on variants of the A* algorithm for pathfinding and decision-making. In the following section, we first introduce A* and then describe how it is adapted within different agent architectures used in level validation.

2.5 AI Agents for Automated Playtesting

AI agents play a crucial role in modern PCG research, acting as automated playtesters. They simulate player behavior to evaluate generated content on various criteria, most notably playability, but also potentially estimating difficulty, identifying exploits, or verifying if specific mechanics are triggered or even required

to trigger [8, 9]. For platform games like Super Mario Bros., pathfinding algorithms are central to creating agents capable of navigating levels [6, 5].

2.5.1 The A* Search Algorithm

A* (pronounced "A-star") is a widely used pathfinding and graph traversal algorithm known for its performance and optimality [31]. It operates as a best-first search, meaning it explores the search space by prioritizing nodes that appear most promising. For each node (representing a state in the search space), A* calculates an evaluation function, typically denoted as $f(n)$:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost incurred to reach node n from the starting node.
- $h(n)$ is a *heuristic estimate* of the cost to reach the goal from node n. It serves as a form of informed guesswork, guiding the algorithm toward promising paths and helping it avoid unnecessary exploration. For example, in a grid-based environment, a common heuristic is the *Manhattan distance* (the sum of the horizontal and vertical distances to the goal), which provides a quick and reasonable estimate of the remaining effort. To ensure that A* finds the shortest possible path, $h(n)$ must be *admissible*—that is, it must never overestimate the true cost to reach the goal.

A* maintains a priority queue of nodes to visit, ordered by their $f(n)$ values, always expanding the node with the lowest f score first. A*'s efficiency and its property of finding optimal paths (with an admissible heuristic) make it a standard choice for pathfinding in grid-based environments and other discrete state spaces [31].

2.5.2 Applying A* to Super Mario Bros.

Applying A* to Super Mario Bros. is non-trivial due to the game's dynamics. The state space (n) cannot simply be Mario's (x, y) tile coordinates. A more complete state must include Mario's current velocity (horizontal and vertical), his power-up status (small, big, fire), the state of nearby enemies or other dynamic elements, and potentially the remaining time, all managed by a forward model.

The forward model is a simulation of the game's logic: given a current game state and a combination of inputs (actions), it predicts the resulting future state. It allows the agent to explore hypothetical futures by applying actions and observing their consequences. Importantly, it can update and clone game states, enabling A* to search through possible futures without altering the actual game.

Actions correspond to combinations of button inputs (left, right, jump, speed) applied over a short time interval. Although technically any combination of inputs is possible, only a subset of them leads to meaningful or valid transitions (e.g., trying to jump while mid-air has no effect). Instead of simulating all combinations blindly, validity can often be checked through simple state conditions (e.g., whether Mario is on the ground)

Designing an effective heuristic $h(n)$ is also crucial. A simple Euclidean or Manhattan distance to the goal position is often insufficient, as it ignores obstacles, gaps, and the complexities of jump trajectories. More sophisticated heuristics might consider the horizontal distance while penalizing height differences or incorporating reachability analysis, as seen in the agents discussed below.

Issues with Determinism

Before detailing the specific agent architectures, it is important to address the topic of determinism in the Mario AI Framework. Although all agents in this framework are designed to be fully deterministic, i.e., given the same level and starting conditions, they should behave identically; this determinism is not always preserved in practice. Two probable sources of non-determinism are: (1) how much CPU time the agent receives from the Java Virtual Machine (JVM), and (2) a simplistic and flawed implementation of time synchronization between frames in the original framework. As a result, agents can sometimes behave inconsistently on the same level, winning a level in one run and losing it in another.

The Slim Forward Model

We now turn to the agent architecture used in this work. The *Slim forward model* developed by David Šosvald [6], was created to address severe performance bottlenecks in the original Mario AI framework's forward model. Its primary goal was efficiency, achieved through several key optimizations:

- Separating static (unchanging) data from dynamic (state-dependent) data to minimize cloning overhead.
- Removing all graphics-related data and processing, which are unnecessary for agent planning.
- Reimplementing level representation and sprite handling for better memory layout and access.
- Utilizing optimized cloning techniques and streamlining update logic.

The result was a forward model dramatically faster than the original, achieving approximately 300× faster cloning and 2.5× to 3.5× speed-up for state advancement. This performance improvement is fundamental to the capabilities of the MFF agents described next.

Baumgarten's A* Agent

One of the most influential AI agents developed for Super Mario Bros. is the A*-based agent created by Robin Baumgarten, which won the Mario AI Competition in 2009 [5]. This agent became a de facto standard and has been widely used in subsequent research, including the original papers for TOAD-GAN [7], DagstuhlGAN [8], and MechanicsGen [9], primarily for evaluating the playability of generated levels. Despite its success and widespread adoption, Baumgarten's agent combined with the original framework's inefficient forward model, suffers from limitations. It can be computationally expensive on complex

levels, as cloning and updating the state are time-consuming and cause the exploration of the large state space to proceed slowly. Furthermore, it is not perfectly complete; there are known instances of solvable levels that the agent fails to complete, partly due to these performance constraints limiting search depth, as well as potential issues with its pruning strategies or heuristic limitations [6, 10]. These shortcomings motivated the development of the Slim forward model and subsequent improved agents.

The MFF A* Agent

The MFF A* agent, presented in *Super Mario A-Star Agent Revisited* [6], was developed specifically to overcome the limitations observed in Baumgarten's approach. It leverages the Slim forward model for significantly improved performance. It represents a re-engineering and optimization of the A* algorithm tailored for Super Mario Bros., incorporating techniques like efficient state caching (using hash tables instead of lists) and optimized action pruning. Benefitting from both the faster model and extensive parameter tuning, the primary goals and achievements of the MFF A* agent are better performance (faster execution speed) and improved completeness (a higher success rate) compared to the Baumgarten's agent baseline, even when Baumgarten's agent is also run with the Slim model.

The MFF A* Grid Agent

A notable variant used extensively in this thesis is the *MFF A* Grid* agent. This specialized version, also developed by Šosvald [10] and built upon the Slim forward model, further enhances efficiency by incorporating a static pathfinding pre-processing step. It first uses a fast grid-based search (ignoring dynamic elements like enemies) directly on the level's tile representation to find an initial path; we refer to this step as the *Grid Search*. This static path information is then used as a domain-specific heuristic to guide the subsequent full A* search using the agent, which does consider game dynamics and physics. By focusing the dynamic search along the promising corridor identified by the static Grid Search, MFF A* Grid can often find solutions much faster and by exploring far fewer nodes than the standard MFF A* agent, especially on levels with relatively clear paths.

The agent's behavior is influenced by several parameters, including the `searchSteps` and `horizontalJumpBoost` settings. The `searchSteps` parameter, common to both MFF A* and MFF A* Grid agents, controls how many simulation frames the agent advances per chosen action. Higher values allow the agent to explore the search space more quickly, making it more efficient, but at the cost of reduced maneuverability. Lower values enable finer control over movement but increase the computational burden. The `horizontalJumpBoost` parameter, specific to the MFF A* Grid agent's static Grid Search phase, increases the maximum jump distance by simulating additional running momentum before a jump. Although this mechanism provides a way to handle exceptionally wide gaps if necessary, in practice, no levels in the test set presented by Šosvald [10] required the use of horizontal jump boosting.

While this reliance on the initial static path introduces a theoretical risk (deviating significantly to handle dynamic obstacles could be penalized heavily by

the heuristic, potentially increasing search cost), failures due to this limitation were not observed on the extensive 1015-level test set used in the initial work. The `BND` (Best Nodes evaluated) parameter configuration of the MFF A* Grid agent, in particular, achieved a 100% win rate while demonstrating the best overall performance profile. By providing a highly reliable and efficient means of automated playtesting, the MFF A* agent and especially the MFF A* Grid (BND) agent serve as the proposed superior alternatives for level validation in this thesis, enabling a re-evaluation of prior work that relied on the potentially less robust Baumgarten's agent.

3 Related Work and Study Selection

Super Mario Bros. serves as a standard benchmark for Procedural Content Generation (PCG) research, particularly for level generation, as demonstrated in numerous studies [32, 19, 33, 34, 29, 35]. Evaluating the completability of generated levels is a crucial step, often accomplished by simulating AI agents attempting to complete them. For many years, the A*-based agent by Robin Baumgarten [5] was the most commonly used tool for this purpose in the Mario AI research community.

The research landscape for Super Mario Bros. AI and PCG has heavily relied on the evolving Mario AI framework. Originating around the time of the competitions hosted by Togelius and Karakovskiy [5, 15], it provided a common platform for agent and generator development. Subsequent enhancements were made by various researchers, including work by Khalifa et al. integrated into later iterations [9, 19, 33]. A significant recent contribution, foundational to this thesis, comes from Šosvald et al. [6, 10], who introduced a highly optimized Slim forward model alongside the efficient MFF agents. These developments directly addressed critical performance bottlenecks and completeness issues observed in earlier agents and framework versions, enabling more robust and efficient level evaluation.

As established in the previous chapter (see Section 2.5.2), Baumgarten's A* agent, despite its historical significance, has known limitations regarding its efficiency and the ability to solve completable levels [6]. A core objective of this thesis is to examine the consequences of these limitations by revisiting three prior studies that relied on Baumgarten's agent for validation.

However, the influence of Baumgarten's agent extends well beyond the scope of these three studies. Numerous other works have employed it in various ways. For instance, several studies focusing on Markov chain-based level generation by Snodgrass et al. [29, 30, 36] used the agent primarily for post-generation completability assessment, similar to the approach in TOAD-GAN. Others, like Kerssemakers et al. with their *Procedural Procedural Level Generator Generator* [34], integrated the agent's performance directly into the fitness function of an evolutionary algorithm to guide level creation, similarly to the agent-in-the-loop methods found in DagstuhlGAN and MechanicsGen. Automatic generation approaches using graph grammars [35] also relied on this agent for validating the completability of their outputs. These examples further underscore the agent's widespread use and highlight the potential impact of its limitations across a large portion of previous research in Super Mario level generation.

3.1 Selection of Studies for Re-evaluation

The selection of studies for re-evaluation focused on identifying influential or representative works where the agent's role, either in validation or directly within the generation process, suggested its limitations could have significantly affected the results. Specifically, we looked for studies where Baumgarten's agent was used either for post-generation completability assessment or as an integral part of

the generation algorithm (e.g., in fitness evaluation). Based on these criteria, the following three studies were chosen as the focus of this thesis.

3.1.1 DagstuhlGAN

The first study selected is **Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network**, referred to here as DagstuhlGAN [8]. This paper was chosen for several key reasons. Firstly, it used Baumgarten's agent not just for the final validation, but during the level generation process itself, specifically within the fitness function guiding the Latent Variable Evolution (LVE) powered by CMA-ES. A flawed agent guiding the evolutionary search could significantly skew the types of levels generated. Secondly, with 315 citations according to Google Scholar (as of April 2025), it represents one of the most cited works in Mario level generation explicitly using Baumgarten's agent, suggesting its findings have had considerable influence. Re-evaluating DagstuhlGAN with the MFF A* Grid agent allows us to investigate how a more capable agent might alter the evolutionary trajectory and the characteristics of the resulting levels.

3.1.2 TOAD-GAN

Given the selection of DagstuhlGAN, which utilizes Generative Adversarial Networks (GANs), **TOAD-GAN: Coherent Style Level Generation from a Single Example** [7] presented itself as a natural second choice. While also employing a GAN-based approach (specifically an adaptation of SinGAN), TOAD-GAN used Baumgarten's agent solely for post-generation validation to report the completability percentage of its generated levels. This provides a useful contrast to DagstuhlGAN. By simply re-evaluating TOAD-GAN's output with the MFF A* Grid agent, we can isolate the impact of the improved agent purely on the assessment of completability of levels, without the confounding factor of the agent influencing the generation process itself. This serves as a clearer baseline demonstration of how the original study might have underestimated the completability of its generated content due to the validator's limitations.

3.1.3 MechanicsGen

Finally, **Intentional Computational Level Design**, referred to here as MechanicsGen [9], was selected. Like DagstuhlGAN, this work integrates the agent directly into the generation loop, using evolutionary algorithms (FI2Pop and CMElites) guided by agent simulations. Its specific goal is generating scenes that *require* specific mechanics (like high jumps or enemy stomps), which makes the agent's capabilities particularly important. MechanicsGen used Baumgarten's agent both as a 'perfect' agent (to check for completability) and as a basis for 'limited' agents (unable to perform the target mechanic). The potential for Baumgarten's agent to miss alternative paths that bypass the intended mechanic, or to mislead the generation, seemed significant. Therefore, replacing it with the MFF A* agent held considerable potential to improve the system's ability to generate levels that truly fulfill the specified mechanical constraints, making it a compelling candidate for re-evaluation and enhancement.

3.2 Summary of Scope

By replicating the methodologies of these three distinct studies—TOAD-GAN (GANs, validation-only), DagstuhlGAN (GANs+LVE, agent-in-the-loop), and MechanicsGen (Evolutionary Search, agent-in-the-loop for constraints)—and substituting the MFF agents, this thesis seeks to provide concrete evidence on the extent to which reliance on the older Baumgarten agent may have influenced prior findings and demonstrate the benefits of using more advanced agents in current PCG research.

4 TOAD-GAN

This chapter focuses on the first of the three procedural content generation studies revisited in this thesis: *TOAD-GAN: Coherent Style Level Generation from a Single Example* [7]. TOAD-GAN, which stands for **T**oken-based **O**ne-shot **A**rbitrary **D**imension **G**enerative **A**dversarial **N**etwork, uses a hierarchical Generative Adversarial Network (GAN) structure to learn the stylistic and structural patterns present within a single input level. This method generates token-based video game levels, specifically demonstrated on Super Mario Bros. In this context, *tokens* refer to the level's predefined building blocks, such as a specific terrain tile (like a ground block), an enemy, or even empty sky. A key advantage is that TOAD-GAN requires only a single example level for training, which could be particularly relevant in game development, where large datasets of levels tend to be limited because of the high cost of manual design.

The primary motivation behind TOAD-GAN was to overcome the limitations of traditional machine learning approaches in PCG, which typically require substantial amounts of training data and often produce levels that lack a coherent style, appearing as mixtures of different training examples. TOAD-GAN addresses this by adapting the SinGAN architecture, originally designed for single natural image generation, to the discrete token-based domain of level maps.

A key aspect of the original TOAD-GAN study was its evaluation methodology, which included the use of Baumgarten's A* agent to test the playability of the generated levels. This chapter will first examine the methodology behind TOAD-GAN, then detail the replication process undertaken for this thesis, and finally present a comparative analysis of level playability using the original Baumgarten's agent against the improved MFF A* Grid agent.

4.1 TOAD-GAN Methodology

4.1.1 Core Architecture: Adaptation of SinGAN

The foundation of TOAD-GAN is SinGAN, a GAN architecture designed to learn from a single image. It is built on a hierarchy of fully convolutional GANs, each trained at a different spatial scale of the input image (or level in this case).

1. **Multi-Scale Architecture:** TOAD-GAN adopts a hierarchical GAN structure inspired by SinGAN, operating across multiple spatial scales, or resolutions, indexed from N (coarsest/smallest) down to 0 (finest/original resolution). During training, the single example level is progressively downsampled (using a hierarchy-sensitive method) to create smaller versions, $x_N, ..., x_0$. Each scale i has its own generator (G_i) and discriminator (D_i). The discriminator D_i is trained to distinguish between patches of the downsampled real level x_i and patches from the fake output $\tilde{x}_i$ generated by G_i. This multi-scale setup prepares the network for the hierarchical generation.

2. **Hierarchical Generation:** Level generation proceeds hierarchically from the smallest scale N to the original scale 0. It begins by feeding random noise z_N to generator G_N. The output $\tilde{x}_N$ is then upsampled and combined

with new noise z_{N-1} to be processed by the next generator G_{N-1}. This process continues iteratively: at each scale i, the generator G_i receives an upsampled version of the previous output and additional noise z_i, and produces $\tilde{x}_i$, a more refined version of the level.

Importantly, each generated output $\tilde{x}_i$ is simultaneously evaluated by a corresponding discriminator D_i, which is trained to distinguish between real patches from the downsampled input x_i and fake patches from $\tilde{x}_i$. This means that at every stage of the generation hierarchy, adversarial feedback is provided, guiding each generator G_i to produce realistic and coherent level segments appropriate for its scale. This layer-by-layer supervision enforces both local and global coherence throughout the generation pipeline. The final level $\tilde{x}_0$ thus integrates both coarse structure and fine details, accumulated across the scales.

3. **Fully Convolutional Nature:** Both generators and discriminators are composed entirely of convolutional layers. This allows them to operate on inputs (patches) of varying sizes during training and, crucially, enables the generation of new levels of different dimensions by starting the process with a noise map of the desired target size at the coarsest scale N.

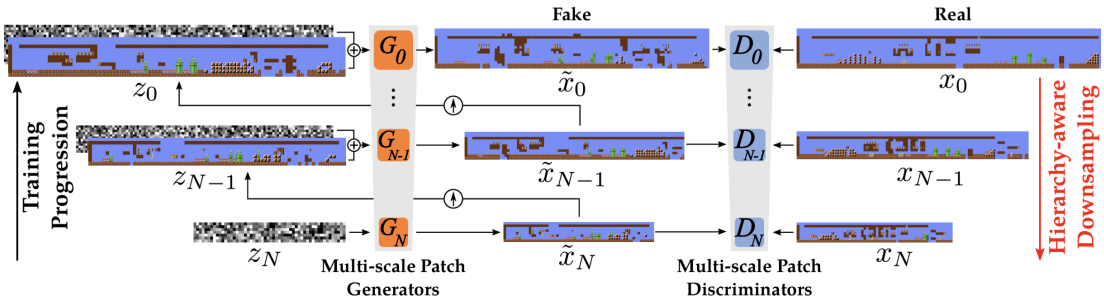


Figure 4.1 Visualization of the Multi-Scale architecture of TOAD-GAN. (Image taken from the original paper and slightly resized to fit the page layout) [7].

4.1.2 Token Representation and Downsampling

Adapting SinGAN from continuous RGB images to discrete, token-based game levels required specific innovations:

1. **Token Encoding:** Levels are represented as multichannel arrays. Instead of RGB, each channel corresponds to a specific tile type (e.g., ground block, question mark block, pipe segment, Goomba). A '1' in a channel at a specific (x, y) coordinate indicates the presence of that tile, while '0' indicates its absence (one-hot encoding).

2. **Hierarchy-Preserving Downsampling:** Standard image downsampling techniques (like bilinear or average pooling) don't work well with token maps, as they tend to eliminate rare but important tokens (e.g., enemies, power-ups, hidden blocks) by averaging them out with more common tokens like sky or ground. TOAD-GAN uses a custom downsampling algorithm based on a predefined token hierarchy. When downsampling, if multiple

tokens occupy the area corresponding to a single token on a smaller scale, only the tokens with the highest rank in the hierarchy are preserved. This ensures that critical gameplay elements are maintained even on a smaller scale (e.g., a Goomba enemy is preserved over an empty sky tile).

4.2 Original Evaluation and Shortcomings

The authors of TOAD-GAN performed an extensive evaluation of their generated levels. Using qualitative assessment, Tile Pattern KL-Divergence (TPKL-Div) [17], and Level Slice Embeddings [7], they demonstrated that their method successfully captured the style and local patterns of the single training examples.

To assess playability, the original study relied on Baumgarten's A* agent, reporting a 65% completion rate on randomly sampled generated levels. Although TOAD-GAN showed strong results in style replication, this reliance on Baumgarten's agent presents a significant shortcoming relevant to this thesis. As discussed previously (Section 2.5.2), this agent is known to be inefficient and can fail on completable levels. Consequently, the reported success rate for playability in the original study might not accurately reflect the true potential of the levels generated, potentially underestimating the number of solvable ones. This limitation motivates the re-evaluation using the MFF A* Grid agent presented in the following sections.

4.3 Replication and Improved Evaluation

For this thesis, the TOAD-GAN framework was replicated following the instructions provided in the original paper and its associated code repository[1] and the additional documentation included in Attachment A. Pre-trained models provided by the authors were used and a new model was trained from a custom level using the documented procedures. The TOAD-GUI tool, a lightweight interface for visualizing, editing, and saving generated levels, was slightly modified to support batch generation and saving of 100 levels at once for easier testing. Additionally, its visualization functionality was adapted to export level images used throughout this thesis.

The core contribution of this thesis regarding TOAD-GAN lies in re-evaluating the playability of its generated levels using the more capable MFF A* Grid agent and comparing its performance directly against the original Baumgarten's agent.

Experimental Setup

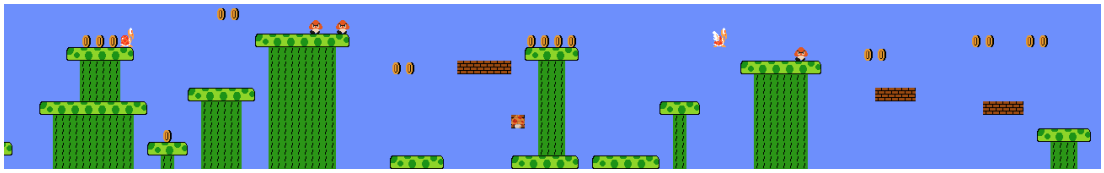
Several sets of 100 levels each were generated using TOAD-GAN models trained on different Super Mario Bros. levels:

- Original Level 1-1 (Overworld)
- Original Level 3-3 (Floating Platforms)
- `krys-1` Level (a custom level used for testing)

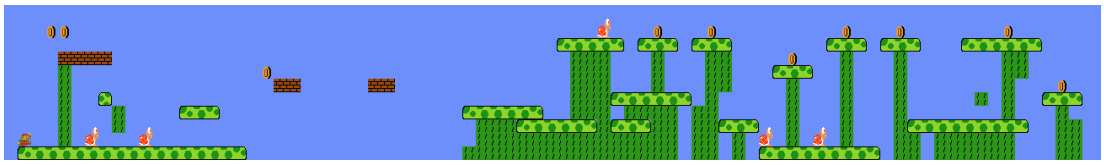
[1] `https://github.com/Mawiszus/TOAD-GAN`

The `krys` level originates from a separate generator that works by stitching together manually created level segments from a large pool. These segments often feature complex parts with little maneuvering space, many enemies, and small maze-like sections requiring Mario to jump repeatedly between ground and platforms to progress. This design intentionally challenges A* search algorithms by significantly increasing the number of states they need to explore compared to more straightforward levels [10].

Additionally, a larger set of 1000 levels was generated using models trained on randomly selected Original levels to provide a broader assessment. Both Baumgarten's A* agent and the MFF A* Grid agent were then run on these generated level sets using the Mario AI Framework. An example of a generated level segment from the Level 3-3 generator, showcasing the stylistic consistency and a level where MFF A* succeeded and Baumgarten's agent failed, is shown in Figure 4.2b.



(a) Part of the original Super Mario Bros. 3-3 level (floating platforms)



(b) Part of the TOAD-GAN generated level trained on 3-3

Figure 4.2 Comparison between the original floating platform level 3-3 (a) and a level generated (b) by a TOAD-GAN model trained on it. While the original Baumgarten A* agent failed to complete the generated level, the improved MFF A* Grid agent successfully reached the end. The generated level convincingly captures stylistic elements like recurring platform layouts and vertical structure.

4.4 Results and Analysis

The comparative evaluation yielded significant differences in measured playability between the two agents. The results are summarized in Table 4.1.

Analysis

Across all test sets, the MFF A* Grid agent consistently demonstrated a substantially higher success rate than Baumgarten's A* agent. This confirms the hypothesis that Baumgarten's agent significantly underestimates the playability of levels generated by TOAD-GAN. Baumgarten's win rate was noticeably lower for levels created by generators trained on more complex level types such as the floating platforms (level 3-3) and `krys-1`, where MFF A* completed roughly 4.4× and 2.4× more levels, respectively.

These findings highlight the limitations of relying on the older agent for evaluation in PCG research. Many levels previously classified as unplayable by the

Table 4.1 Performance of Baumgarten's Agent and MFF A* Grid on various level generators

Level Generator	Total Levels	Baumgarten's Agent Success Rate	MFF A* Grid Success Rate
Original 1-1	100	96%	99%
Original 3-3	100	14%	61%
`krys-1`	100	10%	24%
Random Original	1000	41.8%	77.2%

Note: For 'Original 1-1', manual analysis showed one level was truly uncompletable due to an unjumpable obstacle. Baumgarten's agent also failed on three completable levels (96% success), while MFF A* Grid only failed on the uncompletable one (99% success). Manual checking was done only for this set, not for others.

standard agent used in the original TOAD-GAN paper are, in fact, completable, when tested with a more efficient agent.

Interestingly, there were a few rare instances where Baumgarten's agent managed to complete a level that MFF A* Grid could not. After manually inspecting these levels, it was found that most of them could be completed by MFF A* Grid with a slight `horizontalJumpBoost` applied to the grid search. One specific level required a smaller `searchSteps` value to overcome an obstacle, as Baumgarten's agent uses a default of 2 search steps, while MFF A* Grid uses 3. Both of these parameters are described in more detail in Section 2.5.2. The specific obstacle configuration from this level is illustrated in Figure 4.3.

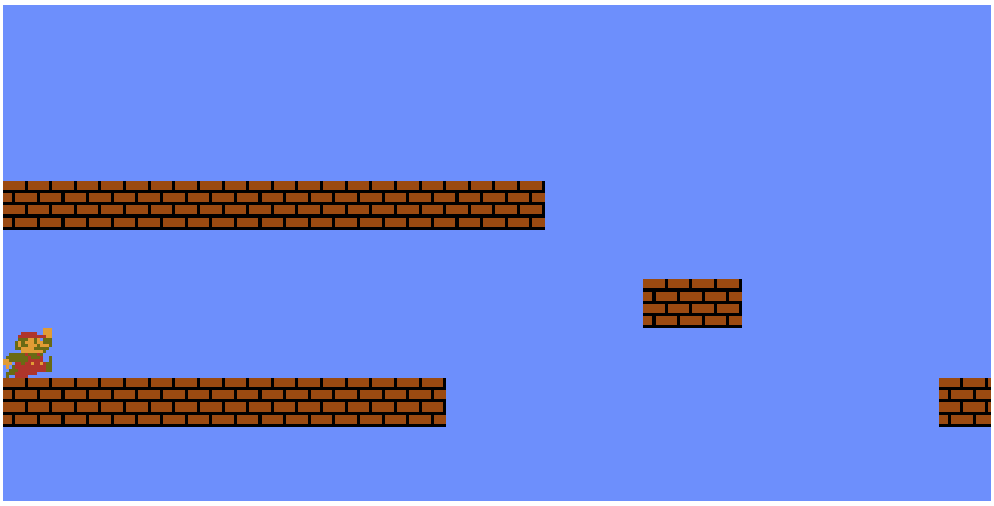


Figure 4.3 The obstacle extracted from the level that Baumgarten's agent could finish and MFF A* Grid could not due to a higher search steps parameter value.

Even though MFF A* Grid performed better overall, a lot of the generated levels were still not completed even with MFF A* Grid (e.g. 49% for 3-3, 76% for the `krys-1` set). This indicates that while TOAD-GAN performs well at capturing style and local patterns from a single example, the generation process itself does not inherently guarantee structural soundness or playability. It is also worth noting that even the MFF A* Grid could have failed on a completable level. However, given the effectiveness of the MFF A* Grid agent and the significant

failure rates, it is likely that a major part of these levels are truly not completable.

4.5 Chapter Summary

The replication done for this thesis confirmed TOAD-GAN's ability to generate varied levels that correspond to the style of the input level. However, the core findings of this chapter relate to the evaluation of playability. By replacing the original Baumgarten A* agent with the more capable MFF A* agent, we demonstrated that the playability of TOAD-GAN generated levels was significantly underestimated in the original study. MFF A* consistently achieved much higher completion rates, revealing that many levels considered unplayable by the previous standard are actually solvable. This shows how important the usage of efficient agents is in PCG research.

5 DagstuhlGAN

Following the replication and evaluation of TOAD-GAN, this chapter investigates the second key study of this thesis: *Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network* [8], referred to as DagstuhlGAN in this thesis, based on the name of its code repository[1]. This work shares TOAD-GAN's goal of generating Super Mario Bros. levels from a single example, but uses a different strategy. Instead of a hierarchical structure, DagstuhlGAN utilizes a standard Deep Convolutional GAN (DCGAN) trained on segments of a single level, and then introduces an evolutionary algorithm—specifically the Covariance Matrix Adaptation Evolution Strategy (CMA-ES)—to search the GAN's latent space for vectors that produce levels with desired characteristics.

The core concept is Latent Variable Evolution (LVE), which treats the trained GAN generator as a fixed mapping from genotype to phenotype. In this setup, the latent vectors (genotypes) are evolved to optimize the output levels (phenotypes) based on specific fitness functions. This approach takes advantage of what the GAN has learned about the level's features, while also steering the generation process toward specific goals, such as playability, or encouraging specific in-game behaviors.

The original DagstuhlGAN work used Baumgarten's A* agent within its evolutionary loop to evaluate level playability and guide the search. This chapter details the DagstuhlGAN methodology, the replication process (including significant technical challenges), the extension involving the MFF A* Grid agent and new fitness functions, and a comparative analysis of the evolutionary outcomes.

5.1 DagstuhlGAN Methodology

The DagstuhlGAN approach consists of two phases: first, training a GAN on level segments, and second, evolving latent vectors using CMA-ES to generate complete levels optimized for certain properties.

5.1.1 GAN Architecture and Training

DagstuhlGAN employs a Deep Convolutional Generative Adversarial Network (DCGAN) architecture, trained using principles from Wasserstein GANs (WGANs) for improved stability. The overall training process is illustrated in Figure 5.1.

- **Training Data:** The GAN is trained on data derived from a single Super Mario Bros. level (Original level 1-1 in the paper). This level is processed by sliding a 28×14 window (representing the screen size of Super Mario Bros.) across it, one tile at a time, creating a dataset of overlapping level segments.

- **Level Representation:** Each tile in a segment is one of 10 possible types (reduced from the number of tiles offered by the Mario AI Framework) and is represented using one-hot encoding. As a result, each segment becomes a

[1] `https://github.com/CIGbalance/DagstuhlGAN`

$10 \times 28 \times 14$ tensor, which is padded to $10 \times 32 \times 32$ to match the network architecture.

- **Network Architecture:** The generator receives a 32-dimensional latent vector z and transforms it into a $10 \times 32 \times 32$ tensor using transposed (fractionally-strided) convolutions and ReLU activations (upscaling). The discriminator uses standard strided convolutions (downscaling) with LeakyReLU activations, and its input is a $10 \times 32 \times 32$ tensor.
- **Generation:** After training, the generator maps a 32-dimensional latent vector z (sampled from a Gaussian distribution or provided by an evolutionary algorithm) to a $10 \times 32 \times 32$ tensor representing a level segment. This output is cropped back to $10 \times 28 \times 14$, and the final level is obtained by applying an `argmax` over the 10 channels at each tile position to recover the discrete tile types.

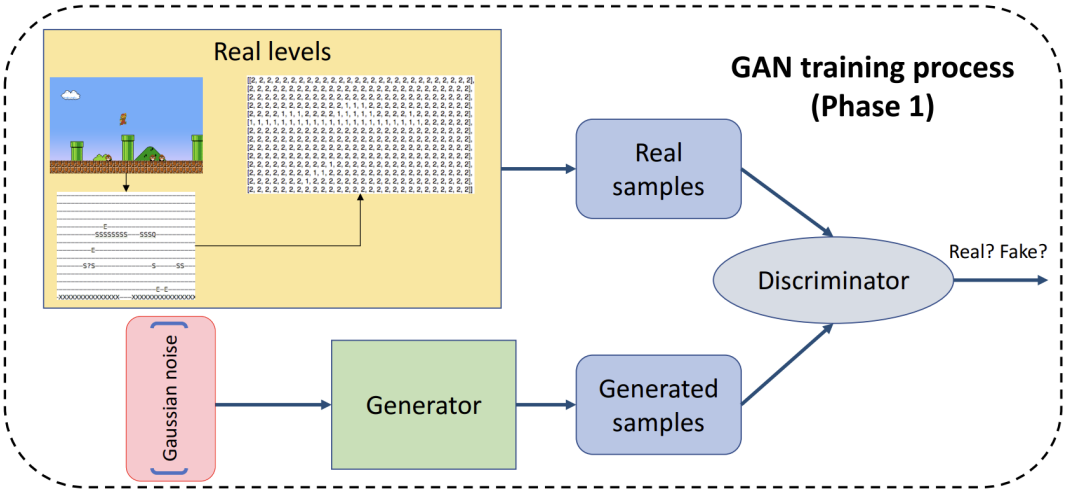


Figure 5.1 The DCGAN architecture used in DagstuhlGAN, showing the generator, discriminator, and the training process. Image taken from the original paper [8].

5.1.2 Latent Variable Evolution (LVE) with CMA-ES

Once the GAN is trained, its generator G is fixed and used within the evolutionary optimization process, as shown conceptually in Figure 5.2. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES), an algorithm for optimizing real-valued vectors in complex search spaces, is employed to search the 32-dimensional latent space. CMA-ES samples latent vectors z, which are passed through the generator to produce level segments $G(z)$. These segments are then evaluated using a predefined fitness function. Based on the evaluation results, CMA-ES adjusts its sampling to guide the search toward latent vectors that return higher fitness, gradually improving the quality of generated levels.

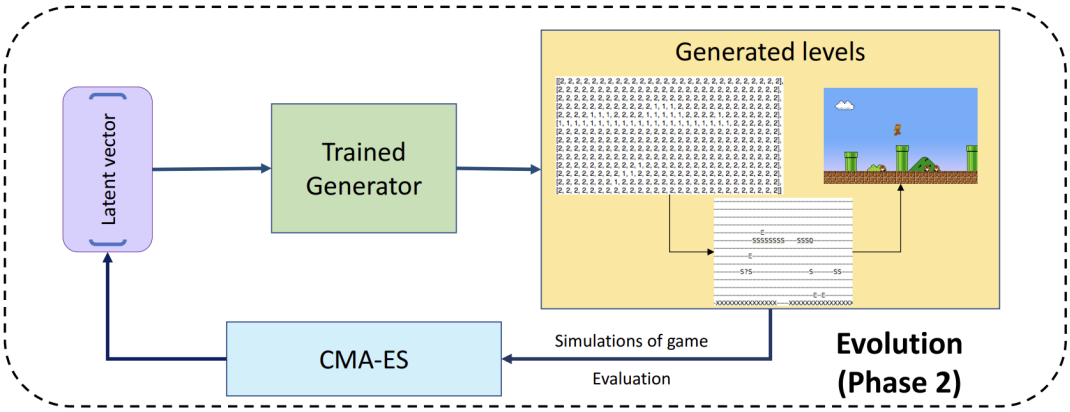


Figure 5.2 Latent vector optimization using CMA-ES and a fixed generator. Image from the original paper [8].

5.1.3 Original Fitness Functions

The original paper explored several fitness functions, including representation-based ones (e.g., targeting specific ground tile percentages) and, more relevantly for this thesis, agent-based ones using Baumgarten's A* agent. The primary agent-based fitness function, aimed at finding levels that were both playable and required a high number of jumps, was used to approximate difficulty. Since CMA-ES minimizes the objective function by default, the fitness functions were negated to convert maximization goals into minimization problems. As a result, lower fitness scores correspond to better matches with the desired level properties. The fitness function was designed accordingly:

Let p be the fraction of the level completed by the agent (x-axis progress, $p \in [0, 1]$) and $\#jumps$ be the number of jumps performed. The fitness F_1 was defined as:

$$F_1 = \begin{cases} -p & \text{if } p < 1 \text{ (level not completed)} \\ -p - \#jumps & \text{if } p = 1 \text{ (level completed)} \end{cases}$$

Minimizing F_1 thus entails completing the level ($p = 1$) and performing many jumps. The original paper also used a variation aimed to minimize jumps while ensuring playability.

5.2 Original Evaluation and Shortcomings

The original DagstuhlGAN paper demonstrated the feasibility of the LVE approach. Representation-based tests showed that CMA-ES could find latent vectors of levels with targeted tile distributions. Agent-based tests showed that the evolutionary process could discover playable levels and optimize for properties such as the number of jumps.

However, the study and its implementation suffered from several limitations identified during the replication process for this thesis:

- **Agent Limitation:** Using Baumgarten's A* agent for fitness evaluation introduces the same drawbacks that were discussed earlier. The inefficiency

of the agent and potential failure on solvable levels could misguide the evolutionary search.

- **Technical Debt:** The project uses one of the first versions of PyTorch (0.3.1), which led to compatibility issues on modern systems. Additionally, the original codebase relied on outdated versions of Python, Java, and the Mario AI Framework, which made it difficult to run on modern systems and incompatible with newer tools like MFF A* Grid. Setting up a functional environment took considerable effort, largely due to the lack of comprehensive documentation.
- **Level Quality:** The generated levels are quite small (28x14 tiles) due to the GAN being trained on screen-sized segments. Visually and functionally, the levels often appeared simplistic and had noticeable issues like broken pipes or seemingly random, purposeless tiles, which limited the perceived quality and gameplay potential.
- **Playability Guarantee:** Similar to TOAD-GAN, the process does not guarantee that evolved levels are playable, even when playability is part of the fitness function.

5.3 Replication and Extension

Before we could focus on replacing the agent and testing new fitness functions, we had to resolve several technical issues involved in running the original DagstuhlGAN code.

5.3.1 Replication Challenges and Setup

The original DagstuhlGAN codebase proved incompatible with modern development environments and libraries. The required PyTorch version (0.3.1) is long deprecated, lacks Windows support, and cannot be installed via standard package managers. Because there was no version for Windows, we had to use either Mac or Linux. We chose to use Mac, which was most likely the operating system the original authors used, as suggested by the presence of `.DS_Store` files in the repository. Compatibility issues also existed with required Java versions and the underlying Mario AI Framework. As detailed in the documentation notes (see Attachments A), a functional environment was eventually established using macOS Monterey, Python 3.6.15 (installed via pyenv), and manually installed archived versions of PyTorch 0.3.1 and torchvision 0.2.0. The Java components, while not tied to any specific IDE, had to be carefully configured in IntelliJ IDEA. We also updated the codebase to be compatible with JDK 17..

In addition to replacing the evaluation agent, this work also explored the effect of training the GAN on a more complex and challenging base level. The original DagstuhlGAN implementation only included a generator trained on the relatively simple Original level 1-1 and provided no built-in support for generating training datasets from other levels. To explore how a generator trained on more intricate source level might influence the evolutionary outcomes and the resulting fitness values, we decided to create a new generator based on the `krys-2` level,

which is characterized by greater structural complexity (as detailed in section 4.3). Consequently, a custom script had to be developed to extract training frames from the `krys-2` level, replicating the original sliding window data preparation approach and ensuring compatibility with the existing training pipeline. This allowed for a direct comparison of evolutionary performance using generators trained on levels of differing complexity.

5.3.2 Framework Update and Agent Replacement

An important step was updating the Java components of DagstuhlGAN to use the newer version of the Mario AI Framework and the MFF agents. Once the framework was updated, Baumgarten's A* agent could be replaced with the MFF agents within the fitness evaluation loop of the CMA-ES process. MFF A* Grid was primarily used in the experiments due to its superior win rate.

To ensure compatibility with the updated framework, several adjustments to the level representation were necessary. The original GAN output produced level segments with a height of 14 tiles, but the grid search of MFF A* Grid expects levels with a height of 16. To resolve this mismatch, two empty rows were added to the top of each generated segment, effectively increasing the height to 16, which is in line with the original game.

Additionally, the horizontal dimensions of the level segments had to be modified to match Mario's spawn position as handled in the old framework. The older version of the framework automatically added starting and ending platforms for Mario specifically, a flat section of ground tiles at the beginning and end of the level. To replicate this behavior, 14 empty columns with ground tiles at the bottom layer were added to the left and 2 to the right of each segment. These modifications are illustrated in Figure 5.3 and ensured that generated levels could be evaluated correctly without causing errors in the updated system.

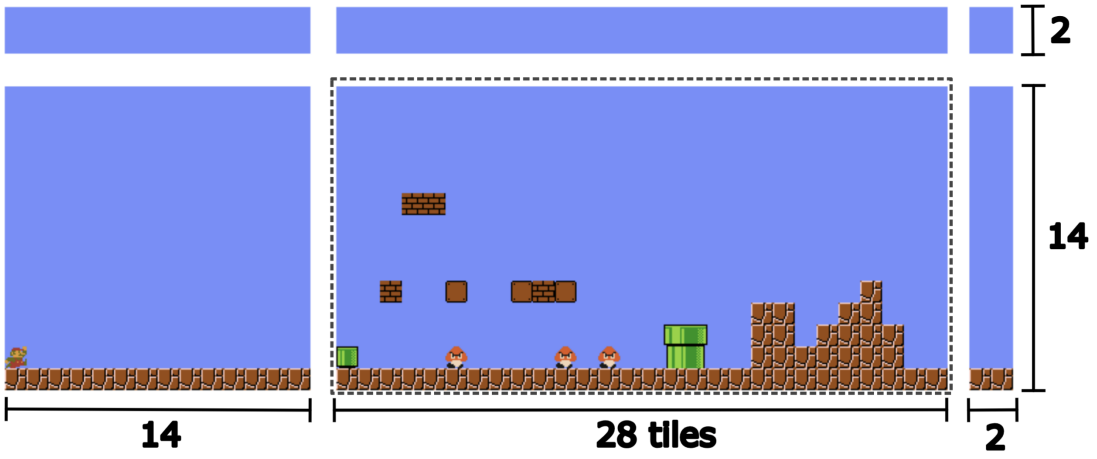


Figure 5.3 Visualization of the changes done to the levels done for compatibility with the new Framework.

5.3.3 New Fitness Functions

Beyond replacing the agent in the original fitness function (F_1) implementation, this thesis explored several new fitness objectives for the CMA-ES evolution. These

were designed to optimize different aspects of gameplay or test the agents' behavior. In all cases, $p \in [0, 1]$ represents the fraction of the level completed by the agent in terms of x-axis progress. CMA-ES minimizes the fitness value.

- **Run-Time Fitness (F_{time}):** Designed to find levels that are playable but take the agent a longer time to complete, potentially indicating more complex or engaging paths. Let t be the time in milliseconds taken by the agent to finish the level if $p = 1$. The fitness is defined as:

$$F_{time} = \begin{cases} -p & \text{if } p < 1 \text{ (level not completed)} \\ -1 - t & \text{if } p = 1 \text{ (level completed)} \end{cases}$$

  Minimizing F_{time} prioritizes level completion ($p = 1$) and then tries to maximize the completion time t.

- **Backtracked Nodes Fitness (F_{nodes}):** Aims to find playable levels that cause the A* search agent to backtrack frequently, suggesting paths that are non-trivial for the agent to discover. Let $\#nodes$ be the number of nodes backtracked by the A* agent during its search. The fitness is defined as:

$$F_{nodes} = \begin{cases} -p & \text{if } p < 1 \text{ (level not completed)} \\ -1 - \#nodes & \text{if } p = 1 \text{ (level completed)} \end{cases}$$

  Minimizing F_{nodes} prioritizes level completion ($p = 1$) and then seeks to maximize the number of backtracked nodes.

- **Grid-Search Jump Fitness (F_{static_jumps}):** This fitness function utilizes a fast, static analysis approach to evaluate level structure, completely bypassing agent simulations. It aims to find levels that possess a path from start to end and feature numerous potential jump sequences along that path.

  Only the Grid Search pathfinding algorithm of MFF A* Grid is applied directly to the tile representation of the level. Solid tiles (ground, blocks, pipes) are treated as obstacles, and passable tiles (sky) as traversable nodes.

  If a complete path ($p_{static} = 1$) is found, the level text analyzer scans this path to heuristically count the number of 'static' jumps ($\#static_jumps$). This count is based on identifying patterns in the path's vertical movement:

  - A jump is counted if the path marker '■' moves upwards for one to four consecutive tiles relative to its vertical position in the preceding column. This heuristic approximates Mario's typical jump height range based solely on level geometry.
  - A jump is also specifically counted if the path traverses a single-tile gap, identified by the pattern of path markers ■■■ directly above solid tiles 'X' separated by a gap (X-X):

    ■ ■ ■
    X - X

This jump counting is a simplification and does not simulate game physics; it serves as a structural proxy for potential agent jumps.

The fitness is then defined as:

$$F_{static_jumps} = \begin{cases} -p_{static} & \text{if } p_{static} < 1 \text{ (path not found)} \\ -1 - \#static_jumps & \text{if } p_{static} = 1 \text{ (path found)} \end{cases}$$

Minimizing F_{static_jumps} prioritizes the existence of a static path to the end and then seeks to maximize the number of heuristically identified jumps along that path.

The primary advantage of this method is its speed; evolution can proceed much faster (100 generations in approximately 10 minutes) compared to agent-based evaluation. The main disadvantage is that it ignores dynamic game elements (with enemies being the primary dynamic entity currently present in the framework) and relies on heuristics for jump detection, meaning the jump counts may not perfectly translate to actual agent performance. Despite these limitations, it proved effective for rapidly evolving levels with quite complex structural features associated with jumping.

Two variants of this analysis were tested: one that counts single-tile gaps as jumps (`Static Analyzer`) and one that excludes them (`Static Analyzer Without One Gaps`). While no formal user study was conducted, levels generated by the `Without One Gap` variant appeared to require more jumps in the real game and were sometimes more stylistically consistent than those produced by the normal `Static Analyzer` variant.

5.4 Results and Analysis

Comparative experiments were conducted by running the CMA-ES optimization for 1000 iterations over multiple independent runs (typically 100) for each fitness function and agent combination. The average progression of fitness was tracked.

5.4.1 Agent Comparison with Original Jump Fitness

We evaluated Baumgarten's agent against the MFF A* Grid agent using the original jump fitness function (F_1), which minimizes negative progress and the number of jumps. On the original generator (trained on the Original level 1-1), the MFF A* Grid agent converged to better fitness values significantly faster, as shown in Figure 5.4. Example levels discovered during this optimization are presented in Figure 5.6.

The total time for running the tests on a Macmini 7.1 was 19.5 hours for the Baumgarten's agent, which corresponds to an average of approximately 5.13 iterations per hour (where 1 iteration equals 1000 generations). In comparison, the MFF A* Grid agent completed the test in 7 hours, generating an average of 14.29 iterations per hour.

Initial tests with the MFF A* Grid revealed issues from inconsistent level heights affecting gap detection, resolved by standardizing heights to 16 tiles.

Baumgarten's agent sometimes got stuck and jumped in place, producing deceptively low fitness scores, while MFF A* Grid proved more robust against such behavior.

When tested on a generator trained on the more complex `krys-2` level, MFF A* Grid evolution with 100 generations completed in 11 hours, compared to 36 hours required by Baumgarten's agent for only 55 generations. The original run was limited to 55 generations as the computational cost became impractically high. This means that MFF A* Grid generated 9.09 generations per hour, while Baumgarten's agent generated only 1.53 generations per hour. The convergence comparison for the `krys-2` generator is presented in Figure 5.5.

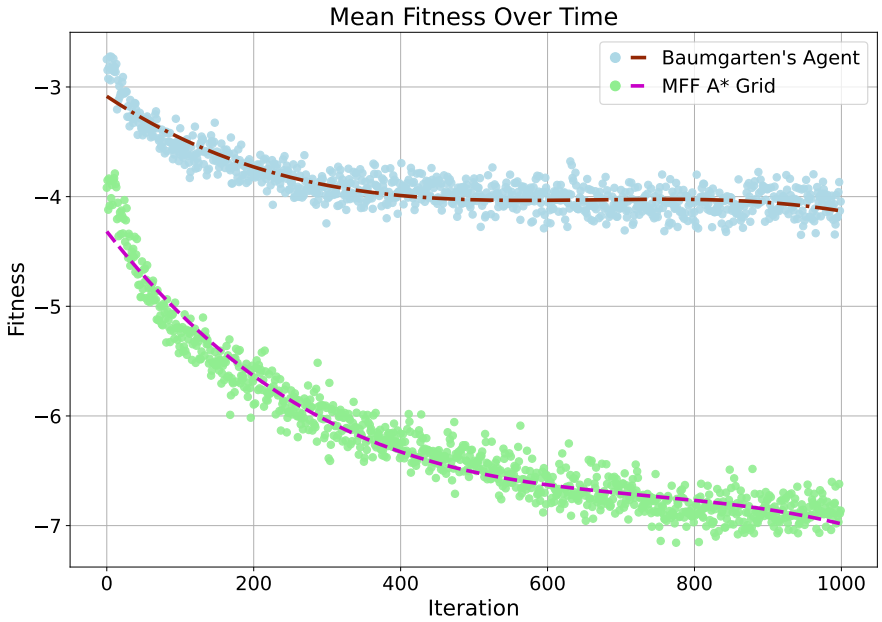


Figure 5.4 Average fitness progression (lower is better) over 1000 CMA-ES iterations using the original jump fitness (F_1) on the generator trained on the Original level 1-1. MFF A* Grid converges faster to better fitness values.

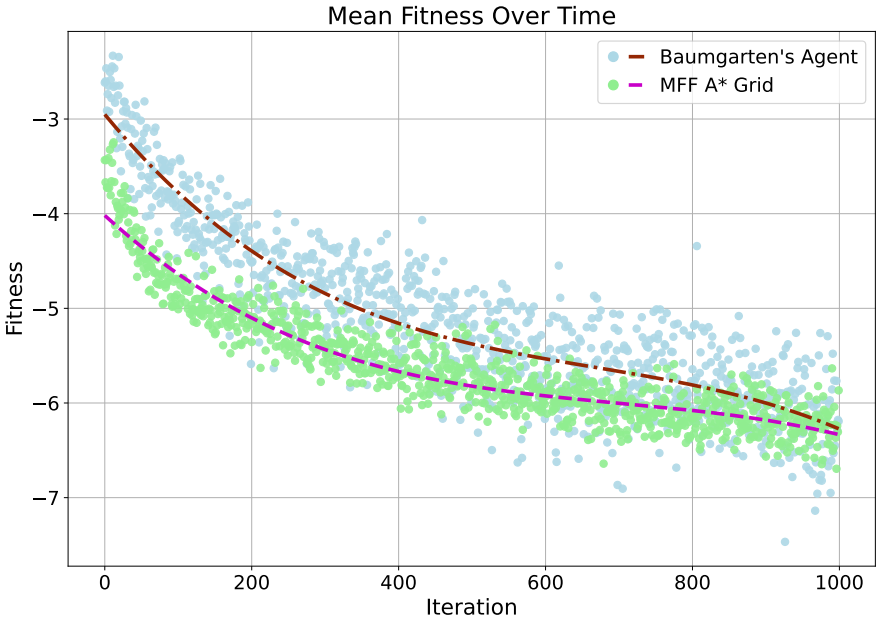
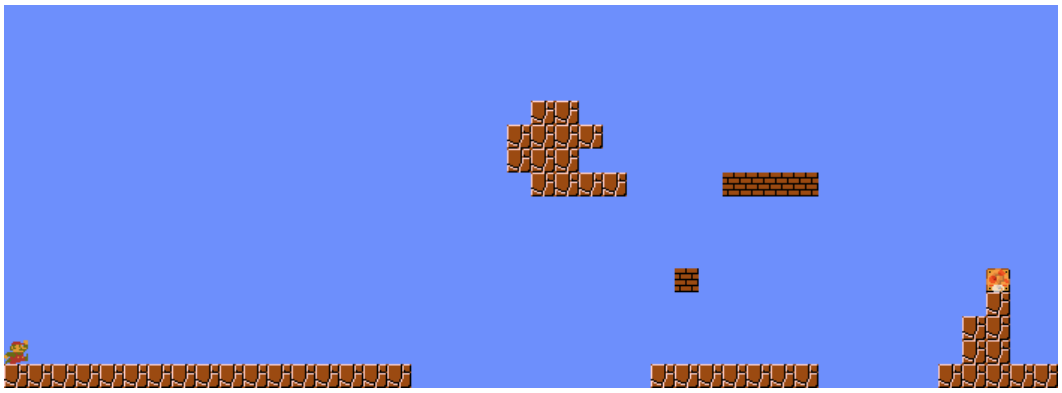
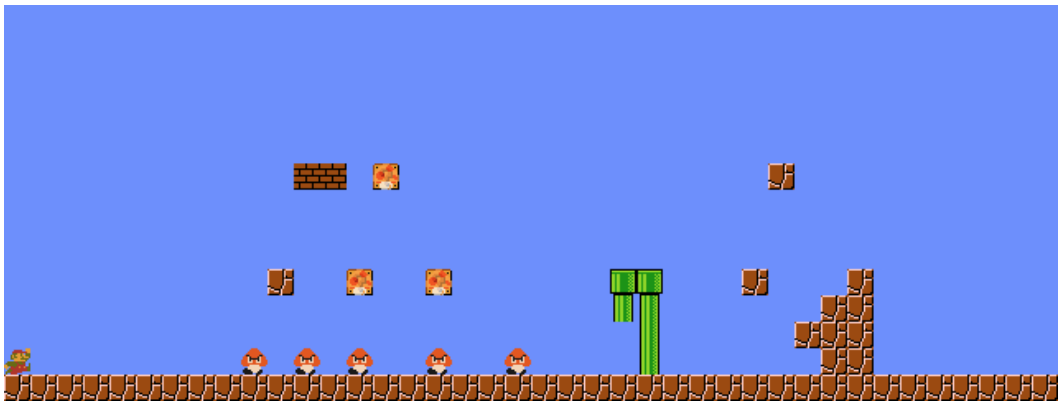


Figure 5.5 Average fitness progression using the original jump fitness (F_1) on the generator trained on `krys-2`. MFF A* Grid converges faster, but final fitness is similar.



(a) Level made by Baumgarten's Agent



(b) Level made by MFF A* Grid agent.

Figure 5.6 Example levels found during optimization with the original jump fitness (F_1). (a) Level generated using Baumgarten's Agent. (b) Level generated using the MFF A* Grid agent. Both levels correspond to some of the lowest fitness values found.

5.4.2 Agent Comparison with New Fitness Functions

Run-Time Fitness

We compared three agents: Baumgarten's agent, MFF A* and MFF A* Grid. Baumgarten's agent was the slowest (it found levels with the longest runtime for the agent). MFF A* had almost a stable runtime and was the most efficient at completing levels (but not at maximizing playtime). It could be assumed that it is difficult to find levels in which the runtime of MFF A* is long. Figure 5.7 illustrates these runtime characteristics, while example levels discovered during this process are shown in Figure 5.8.

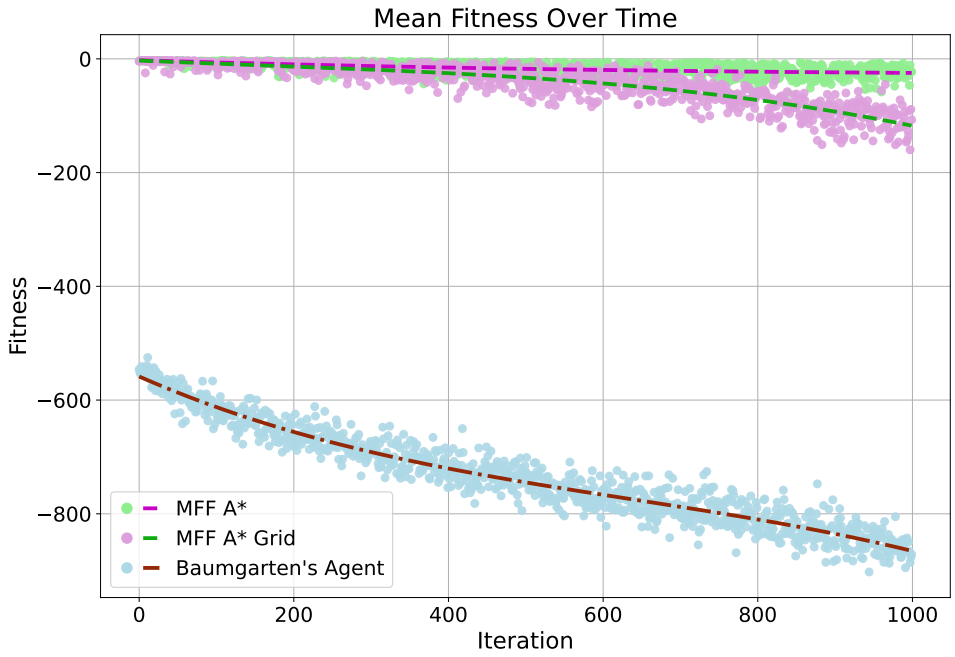
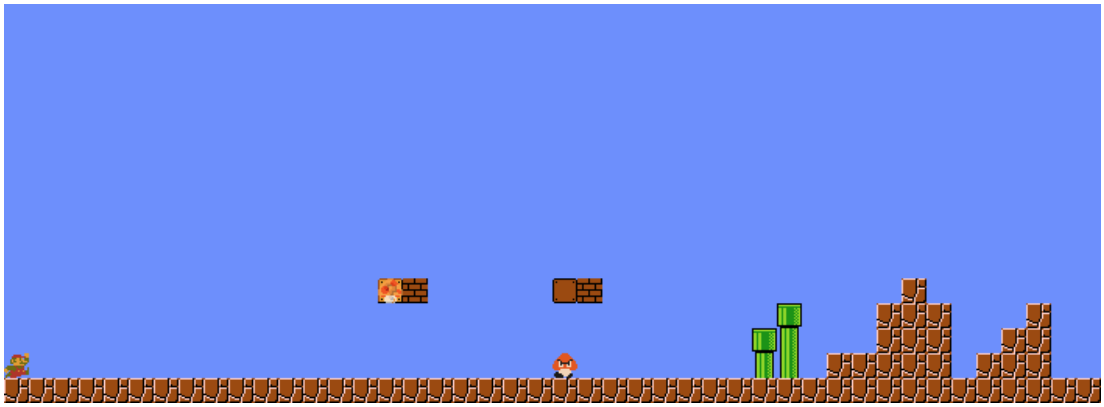
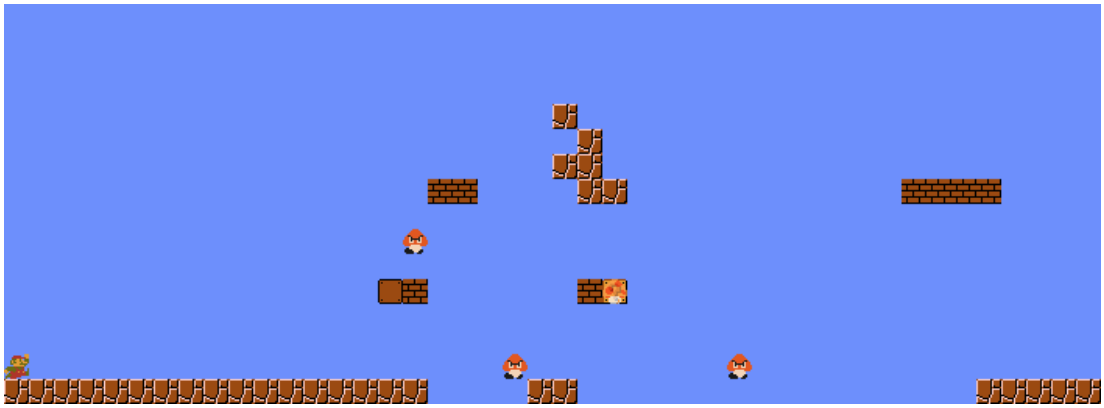


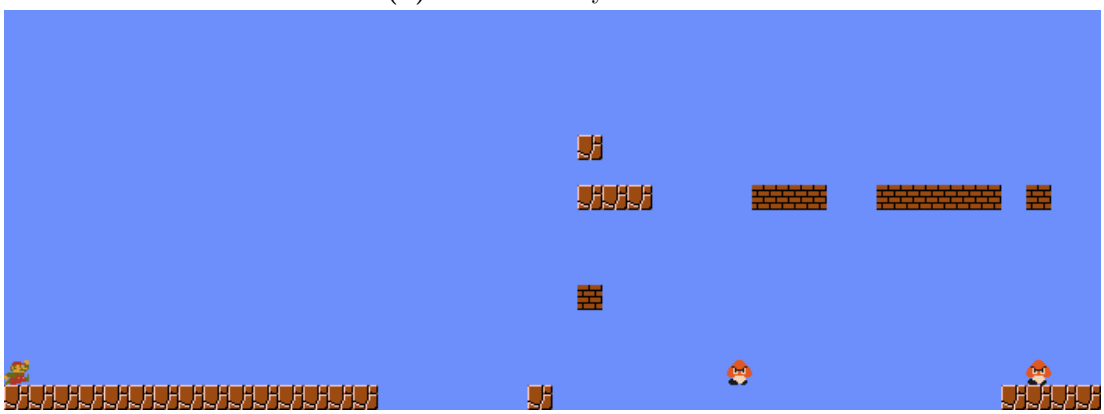
Figure 5.7 Fitness progression of the Run-Time test. Lower values indicate slower level completion. This metric serves as a proxy for testing agents—longer runtimes may suggest weaknesses in pathfinding. Baumgarten's agent is already slow from the first generation and continues to perform poorly. In contrast, MFF A* maintains a stable, low runtime, reflecting higher efficiency, though it likely struggles to maximize runtime due to stronger pathfinding. MFF A* Grid gradually found levels that took a bit longer to complete.



(a) Level made by Baumgarten's Agent



(b) Level made by MFF A*



(c) Level made by MFF A* Grid

Figure 5.8 Example levels with some of the lowest fitness values found during the runtime-based optimization. (a) Level generated by Baumgarten's Agent, which appears relatively straightforward and quick to complete. (b) Level generated by MFF A*, which includes more obstacles or pathing complexity, requiring more time to complete. (c) Level generated by MFF A* Grid.

Backtracked Nodes Fitness

We compared three agents: Baumgarten's agent, MFF A* and MFF A* Grid. Baumgarten's agent reached lower fitness values and hit its limit quickly in this optimization. Its search inefficiency meant it likely failed on more complex levels requiring extensive exploration, thus preventing the discovery of solvable levels with significantly more backtracking and stalling further fitness improvements

(as finding harder solvable levels would decrease the fitness score). In contrast, MFF A* (and MFF A* Grid) continued to find and solve progressively harder levels that involved more backtracking. This sustained improvement, shown in Figure 5.9, demonstrates a more meaningful optimization path, indicating their ability to navigate and complete more challenging search spaces compared to Baumgarten's agent for this metric, even if those situations require more complex pathfinding (reflected in the higher backtracking counts).

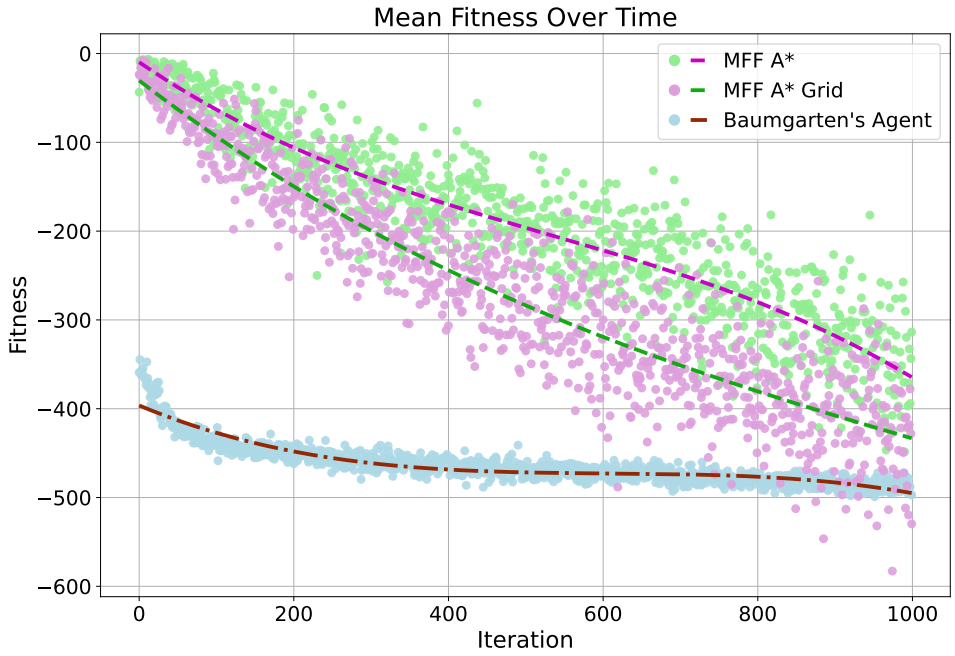
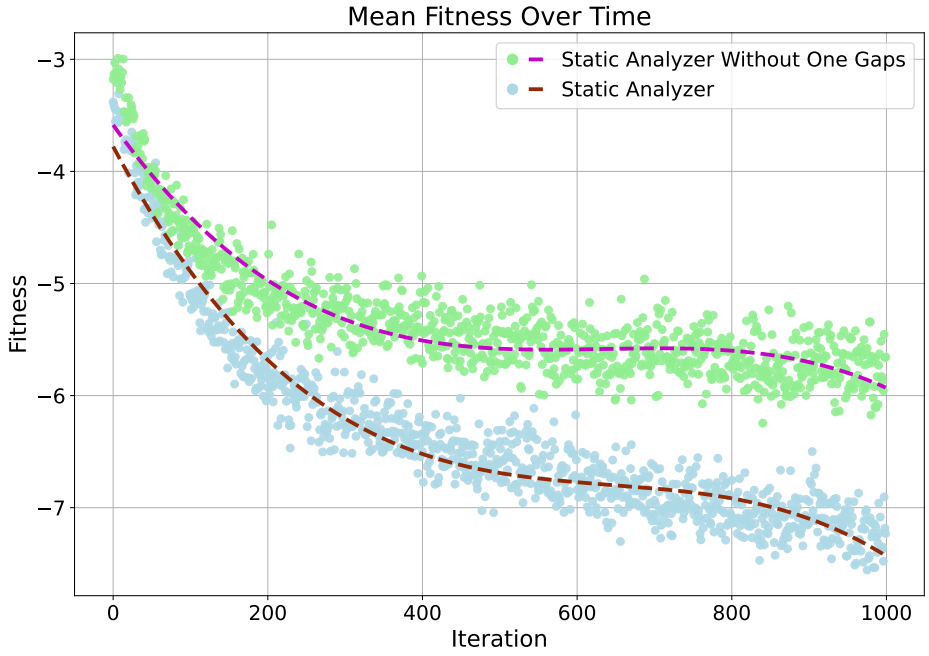


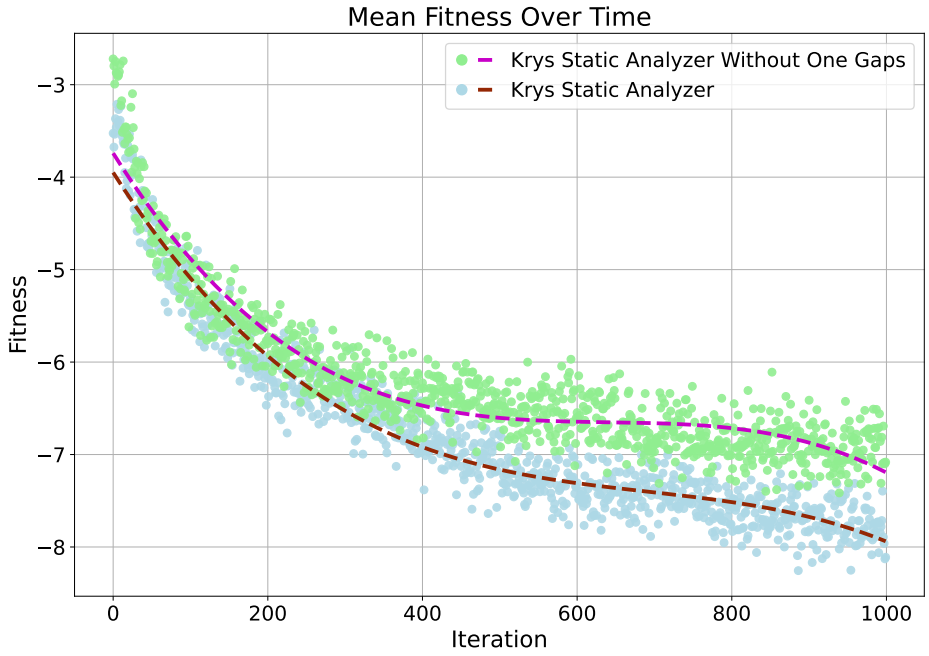
Figure 5.9 Average fitness progression using Backtracked Nodes fitness (F_{nodes}) on the generator trained on the Original level 1-1. Optimization aims for solvable levels with high backtracking (lower scores). Baumgarten's agent reached lower values and hits a limit due to inefficiency on harder levels. MFF A* and MFF A* Grid effectively optimize by finding and solving increasingly challenging levels that induce more backtracking.

Grid-Search Jump Fitness

This static approach proved to be very fast (100 generations in 10 minutes). It successfully evolved levels with many potential jumps based solely on static structure, providing a quick way to explore the level's structural complexity, although it did not consider dynamic elements like enemies. At times, the level generation seems to exploit the heuristic by frequently producing one-tile gaps or two-tile-wide stairs, which in the actual game can be overcome with fewer jumps. The results for both the Original level 1-1 and `krys-2` generators, comparing the two jump counting variants, are displayed in Figure 5.10. Example levels with the lowest fitness values, highlighting the differences between the heuristics and their impact on level structure, are shown in Figure 5.11.

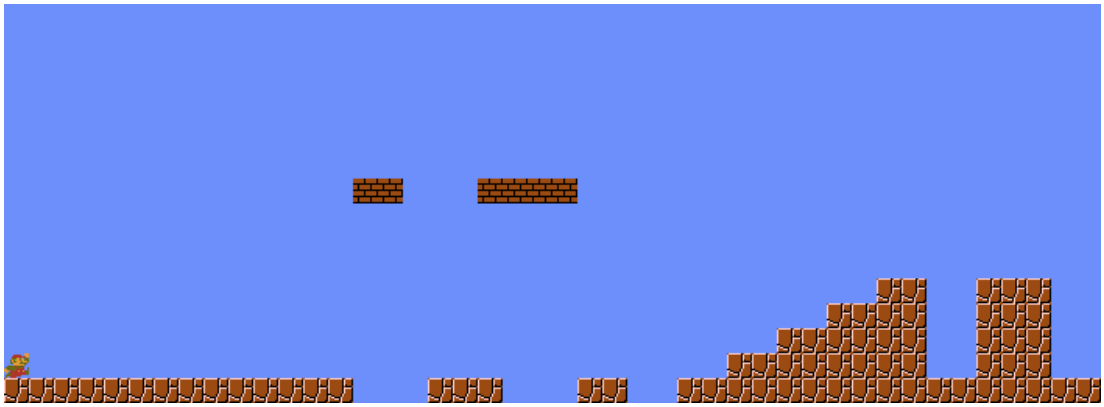


(a) Generator trained on Original level 1-1

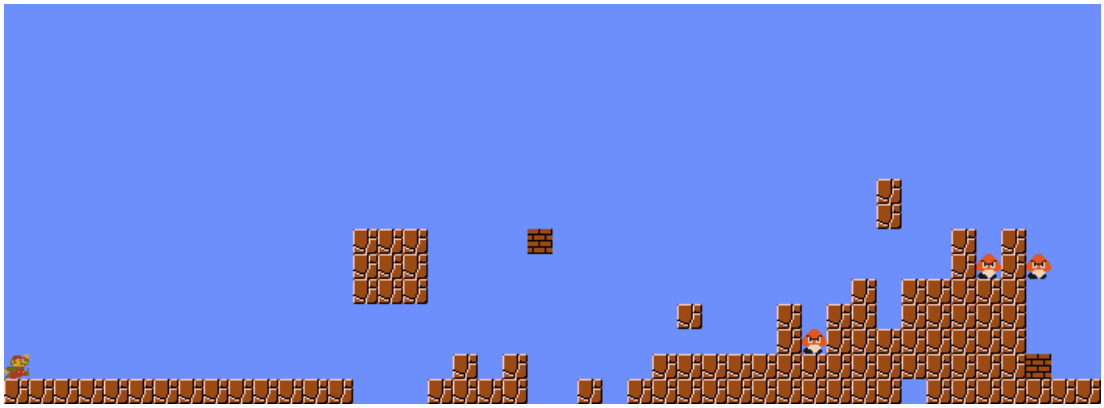


(b) Generator trained on `krys-2`

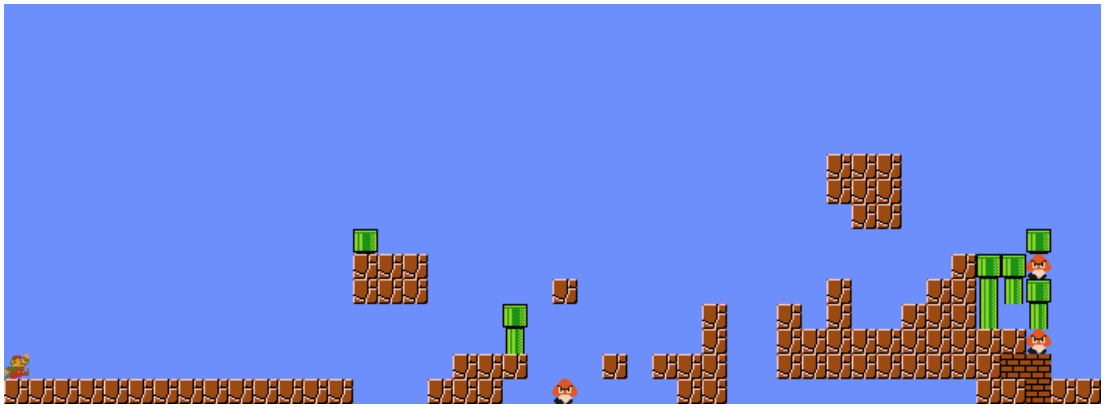
Figure 5.10 Average fitness progression (lower is better) over 1000 CMA-ES iterations using the Grid-Search Jump Fitness (F_{static_jumps}). Both plots compare the standard heuristic (`Static Analyzer`) including single-tile gaps with a variant excluding them (`Static Analyzer Without One Gaps`). Optimization using static analysis is effective and rapid for both the generator trained on (a) Original level 1-1 and (b) the `krys-2` level.



(a) Level made by Static Analyzer Without One Gaps



(b) Level made by Krys Static Analyzer



(c) Level made by Krys Static Analyzer Without One Gaps

Figure 5.11 Example levels with some of the lowest fitness values found using the Grid-Search Jump Fitness. (a) Level evolved by the static analyzer excluding single-tile gaps, trained on the Original level 1-1. While aesthetically pleasing, the level exploits the heuristic by generating a two-tile-wide stair structure that artificially increases the jump count. (b) Level evolved using the standard heuristic on the `krys-2` generator, which takes advantage of one-tile-wide gaps to maximize jump potential. (c) Level evolved with the one-gap heuristic disabled, resulting in a structure where many of the jumps are genuinely required for traversal, illustrating a more meaningful expression of platforming complexity.

5.4.3 Overall Analysis

The results consistently show that MFF A* Grid serves as a superior evaluation agent compared to Baumgarten's agent within the DagstuhlGAN LVE framework. It leads to faster convergence, often toward better or more meaningful solutions, and avoids some of the pitfalls observed with the older agent. Additionally, the MFF A* Grid agent demonstrated significantly faster generation times compared to Baumgarten's agent, making it a more efficient choice for procedural level generation. The experiments also validated the potential of alternative fitness functions, including the highly efficient static grid-search analysis for optimizing structural features.

Training the initial GAN on a more complex level like `krys-2` seemed to result in a generator that produced more diverse levels during evolution compared to the simpler Original 1-1 level, highlighting the importance of the initial training data even when using LVE.

However, a critical limitation remains: the quality of the generated levels. Despite successful optimization according to various fitness metrics, the resulting 28x14 levels often feel simplistic, fragmented, or contain artifacts. While the LVE process works, the underlying GAN generator, trained on small, potentially overlapping segments of a single level, may not capture the necessary high-level structure or variety needed for a truly interesting level design at this scale. Improving the base generator appears to be the key to increasing the practical effectiveness of the LVE approach in this context.

5.5 Chapter Summary

This chapter detailed the significant technical effort required to replicate and modernize the original DagstuhlGAN framework. The core contribution was replacing the Baumgarten A* agent with MFF A* variants during the evolutionary phase and introducing new fitness functions.

Comparative results demonstrated that MFF A* Grid provides faster and often more robust optimization compared to the original agent. New fitness functions, particularly the static grid-search analysis, showed promise for efficient exploration of specific level properties. However, despite the improvements in the evolutionary process, the final generated levels often lacked quality and complexity due to the limitations of the small segment size and the underlying GAN architecture used in the original study.

6 MechanicsGen

This chapter shifts focus to the third and final study addressed in this thesis: *Intentional Computational Level Design* [9], which we will refer to as MechanicsGen. Unlike TOAD-GAN's focus on consistent style or DagstuhlGAN's exploration of latent space, MechanicsGen takes on the challenge of generating small Super Mario Bros. level sections, termed "scenes," that are specifically designed to enforce the use of targeted game mechanics, such as high jumps or stomping on enemies. The core idea is to create level structures where engaging with a specific mechanic provides a clear path forward or advantage, potentially serving as a basis for tutorials or difficulty progression.

MechanicsGen employs search-based procedural content generation, utilizing evolutionary algorithms (specifically Feasible Infeasible 2-Population and Constrained Map-Elites) guided by agent-based simulations. The original study, as with the others examined, relied on Baumgarten's A* agent for evaluating playability and determining if mechanics were potentially used or required. This chapter details the MechanicsGen methodology, identifies significant shortcomings encountered during replication, describes the substantial extensions and improvements implemented in this thesis, and analyzes the resulting impact on performance and level generation outcomes.

6.1 MechanicsGen Methodology

The goal of MechanicsGen is to generate short level scenes (14 vertical slices wide, padded with 3 floor slices on both ends) that are intentionally built to trigger specific mechanics. It explores three distinct search-based approaches: `Limited Agents`, `Punishing Model`, and `Mechanics Dimensions`.

6.1.1 Core Approaches

- **`Limited Agents:`** This technique uses two separate agents simulated on the same generated scene. One is considered a *perfect agent* (ideally, capable of using all mechanics effectively – approximated by Baumgarten's A* in the original work). The second is a *limited agent*, programmatically restricted from performing a specific target mechanic (e.g., unable to run, unable to hold the jump button long enough for a high jump, 'blind' to enemies). The scene is considered successful (satisfies constraints) if the perfect agent can complete it, but the limited agent cannot. This approach uses the Feasible Infeasible 2-Population (FI2Pop) algorithm to evolve scenes toward this constraint.

- **`Punishing Models:`** This approach uses only the perfect agent but simulates it under two different forward models (game physics simulations). The *normal model* behaves as expected. The *punishing model* simulates a scenario where the agent believes it will fail (i.e., die) if it performs a specific mechanic (e.g., stomping an enemy, collecting a coin). The constraint evaluation is similar to `Limited Agents`—the scene is valid if the agent succeeds

under the normal model but fails under the punishing model. This also uses the FI2Pop algorithm.

- **`Mechanics Dimensions:`** This technique uses only the perfect agent and the normal forward model. It employs the Constrained Map-Elites (CMElites) algorithm. After the agent plays through a scene, the playthrough is analyzed to determine which predefined mechanics were triggered (e.g., jump, high jump, stomp, shell kill). These triggered mechanics form the dimensions of the Map-Elites archive. The generated scene (chromosome) is placed into the corresponding cell in the map based on the combination of mechanics used. The goal is to explore the space of achievable mechanic combinations within playable scenes.

6.1.2 Scene Representation and Evaluation

Scenes are represented as sequences of 14 vertical slices sampled from the original Super Mario Bros. levels (via the VGLC [14]), with three columns of floor padding added to each end for safe start and finish areas.

Scene evaluation involves two main parts:

1. **Constraint Satisfaction:** Completability is determined using the agent simulation (originally Baumgarten's agent). The first step is to verify if the scene satisfies the specific requirements for the chosen approach (i.e., perfect agent wins and the limited agent or punishing model loses). If the scene meets these requirements, the constraint is set to 1. If not, the constraint fitness is calculated based on the relative distances traveled by the different agents.

$$\text{constraints} = \begin{cases} 1, & \text{if } A_{\text{perf}} = 1 \text{ and } A_{\text{limit}} = 0 \\ \frac{d_{\text{perf}} - d_{\text{limit}}}{d_{\text{scene}}}, & \text{otherwise} \end{cases} \tag{2}$$

   where A_{perf} and A_{limit} indicate whether the perfect and limited agents completed the level (1 if completed, 0 otherwise), d_{perf} and d_{limit} are the distances they traveled, and d_{scene} is the total scene length. For the Mechanics Dimensions variant, which uses only the perfect agent, constraint fitness is instead defined as:

$$\text{constraints} = \begin{cases} 1, & \text{if } A_{\text{perf}} = 1 \\ \frac{d_{\text{perf}}}{d_{\text{scene}}}, & \text{otherwise} \end{cases} \tag{3}$$

2. **Simplicity Fitness:** If constraints are met, a secondary fitness function is applied to reward scenes with lower visual entropy (fewer different tile types) and higher horizontal consistency (fewer abrupt tile transitions). This encourages clearer and more intentional scene designs. The combined simplicity fitness is defined as:

$$\text{fitness} = 0.2 \cdot (1 - H(X)) + 0.8 \cdot (1 - H(\bar{X})) \tag{1}$$

where $H(X)$ is the entropy of the tile types used in the scene and $H(\bar{X})$ is the entropy of horizontal tile transitions. The latter is weighted more heavily to penalize noisy and inconsistent designs.

6.2 Original Evaluation and Shortcomings

The original MechanicsGen paper presented results showing that all three approaches could generate scenes where the targeted mechanics seemed necessary or possible. However, the replication efforts done for this thesis revealed significant shortcomings.

- **Agent Limitation and Mechanic Optionality:** The reliance on Baumgarten's agent for constraint evaluation proved problematic. As observed in previous chapters, this agent is not exhaustive in its search. Consequently, generated scenes often allowed the target mechanic to be bypassed. The agent might find one path requiring the mechanic, but fail to identify simpler alternative paths. The generated levels, therefore, often made the mechanic optional rather than strictly required for completion.
- **Technical State and Incompleteness:** The publicly available repository[1] was found to be in a poor state. Crucial components, such as the implementations for the `Limited Agents` described in the paper, were entirely missing from the codebase. The elitism mechanism within the FI2Pop implementation did not function correctly, hindering the evolutionary process. Furthermore, the project lacked documentation, making setup and modification challenging.
- **Performance and Usability:** The original implementation had extremely long run times, taking around 10 hours on average for a standard run using 25 CPU cores. Its execution relied on a cumbersome multi-process architecture where the user had to launch multiple separate child processes (runners) manually, communicating via intermediate files, which is inefficient and difficult to manage.

Due to these major technical and performance hurdles, significant rework was required.

6.3 Replication and Extension

Addressing the shortcomings of the original MechanicsGen implementation was a major part of the work for this thesis. The goals were to create a functional and usable framework, integrate a superior agent, and improve performance.

6.3.1 Code Completion and Bug Fixing

Significant effort was invested in completing and correcting the original codebase:

[1] `https://github.com/amidos2006/MarioExperiments`

- **Limited Agents Implementation:** The missing agents with limitations, described conceptually in the paper, were implemented from scratch based on their description, allowing for its proper evaluation.

- **Elitism Mechanism Fix:** The evolutionary algorithms (FI2Pop, CMElites) depend on correctly preserving the best-found solutions (elites) across generations. The original elitism was faulty, sometimes leading to the loss of good solutions, particularly worsened by the agent non-determinism (discussed in section 2.5.2). This was corrected to properly maintain the best-performing individuals according to their fitness and constraint satisfaction status.

6.3.2 Agent Integration and Optimization

- **MFF A* Integration:** The MFF A* agent was integrated into the framework as an alternative to the Baumgarten's agent for all simulation and evaluation tasks. We chose the simple MFF A* over the MFF A* Grid, as the grid-based pathfinding could lead the agent to obstacles that the limited or punished agents could not overcome, potentially altering the generation process.

- **MFF A* Plus Development:** During performance profiling, specific bottlenecks within the MFF A* agent and the Slim forward model were identified. To further reduce initilization and simulation time, several optimizations were implemented, creating a faster version referred to as *MFF A* Plus*. These included pre-calculating frequently used values, avoiding unnecessary object cloning, and conditionally skipping irrelevant code paths (e.g., specialized logic for entities like Bullet Bills when none were present in the current level segment).

6.3.3 Addressing Agent Non-Determinism

A critical issue observed during longer runs was the apparent non-determinism of the agents, including Baumgarten's agent. Occasionally, an agent would solve a scene in one evaluation but fail it in a subsequent one (or vice versa), even though the level itself hadn't changed. This could lead to two main problems:

- **Generation Stalling:** The fitness of the entire population could collapse to zero if the current elite, upon re-evaluation, was suddenly considered unsolvable by the agent, effectively restarting the search.

- **Misleading Evolution:** A scene might be incorrectly classified as satisfying the constraints (e.g. perfect agent wins, limited agent fails) simply because the limited or punished agent got 'unlucky' on that specific run and failed for reasons unrelated to the mechanic limitation. The original paper attempted to moderate this by always storing the lowest constrain fitness of a chromosome between generations, but this did not fully prevent the issue.

Three mechanisms were introduced to combat this:

1. **Robust Elitism:** The corrected elitism mechanism now permanently saves the best validated elite solution. It is not re-evaluated in subsequent generations unless a new chromosome achieves an even better validated fitness score. This prevents the loss of the best-known solution due to a single failed re-evaluation.

2. **Elite Re-Testing:** Before a new chromosome can replace the current elite, it undergoes an additional validation step (re-simulation) to increase confidence that its better performance was not just a result of agent non-determinism on a single run. This reduces the probability of promoting misleadingly good levels.

3. **Punishing Model Event Verification:** Specifically, for the Punishing Model approach, event detection was integrated into the simulation of the perfect agent. In addition to checking whether the agent succeeded normally and failed under the punishing model, the system now also verifies if the perfect agent actually triggered the specific punished mechanic during its successful playthrough. This provides an additional layer of filtering against scenes solved due to non-determinism bugs (where the punishing model might have failed accidentally) and actively steers the evolution toward generating scenes where the mechanic is used by the perfect agent.

6.3.4 Multithreading and Caching

To address the severe performance bottlenecks and usability issues, we implemented the following:

- **Multithreading Architecture:** The cumbersome multiprocess, file-based communication system was completely replaced with a multithreaded architecture. The main program now manages a pool of worker threads, distributing scene evaluation tasks and collecting results directly via shared memory (arrays and object references), eliminating file I/O overhead and simplifying execution.

- **Agent and Model Adaptation:** To support this threading architecture, it was necessary to modify the implementation of the MFF A* agent and the Slim forward model. Both were originally designed with single-threaded assumptions and required refactoring to ensure thread safety and avoid shared mutable state between threads (e.g., adjustments to `MarioUpdateContextSlim.java` and related components).

- **Result Caching:** A caching mechanism was implemented. Before evaluating a newly generated scene, the system checks if an identical scene (based on its tile representation) has already been evaluated during the execution. If so, the cached result (fitness and constraint values) is retrieved instantly, avoiding redundant agent simulations, which are the most time-consuming part of the process. This is particularly effective because the evolutionary process can generate identical scenes from different parent chromosomes or mutations.

6.4 Results and Analysis

The implemented changes resulted in substantial improvements in performance and provided insight into agent behavior.

To ensure consistency and reproducibility, all experiments were conducted on the MetaCentrum computational grid. A CPU with a SPECfp2017 benchmark score of 13.1 was used across all runs, standardizing hardware conditions and ensuring equivalent performance. Each test utilized 25 cores. This controlled environment allowed fair comparisons of runtime and resource usage between the different agents.

6.4.1 Agent Performance Comparison

The average time to complete a standard 1,000-generation run was first measured using the framework, without caching and multithreading, to ensure a clearer comparison of the agents. The average was always calculated from 5 runs each of the 3 `Limited Agents` tests and 5 runs each of the 6 `Punishing Models` tests, resulting in a total of 45 runs:

- **Baumgarten's agent:** Averaged approximately 10 hours and 49 minutes, consistent with the implications of the original paper.
- **MFF A*:** Averaged significantly less, around 3 hours and 30 minutes.
- **MFF A* Plus:** Showed further improvement, averaging approximately 2 hours and 5 minutes.

This clearly demonstrates the inherent speed advantage of MFF A* over Baumgarten's agent for this task, with the optimizations of MFF A* Plus providing further gains.

6.4.2 Improved Agent Configuration

Despite using the improved MFF A* agent, manual inspection of generated scenes (e.g., for the Coin Punishing Model) sometimes still revealed paths that allowed completion without triggering the intended mechanic. Further investigation suggested this might be related to the agent's search parameter `searchSteps` (similar to the problem mentioned in TOAD-GAN), which controls how many simulation steps the agent looks ahead per search node expansion (see Section 2.5.2 for details). Reducing the `searchSteps` parameter to 1 fixed the problem and the agent is now able to find the alternative paths that do not trigger the event. Figure 6.1 shows an example of a level containing such an obstacle, discovered during generation with `searchSteps` set to 3.

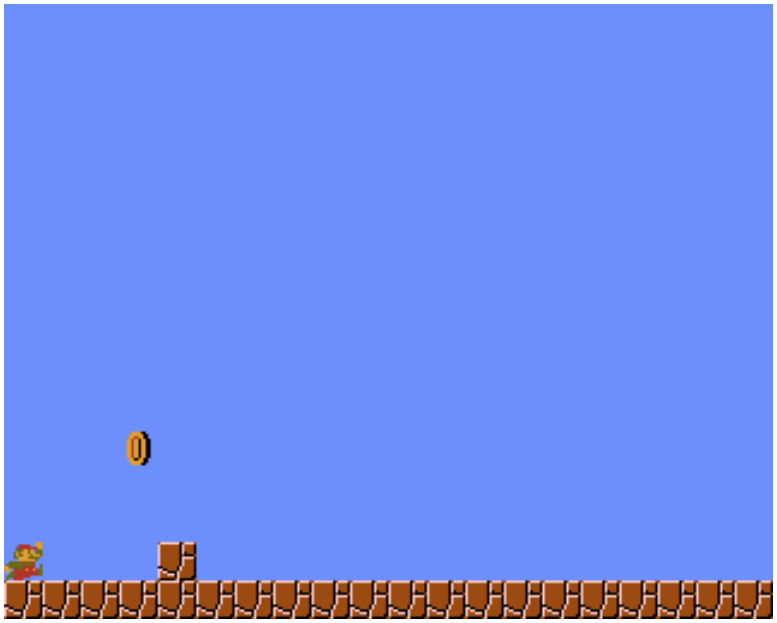
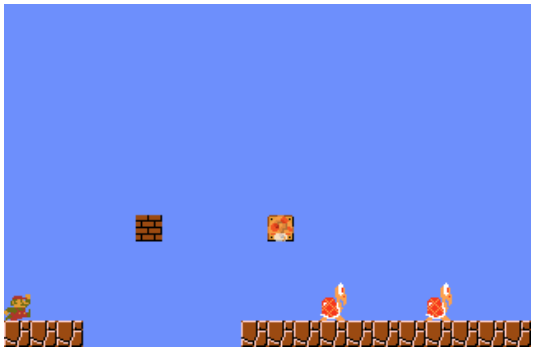


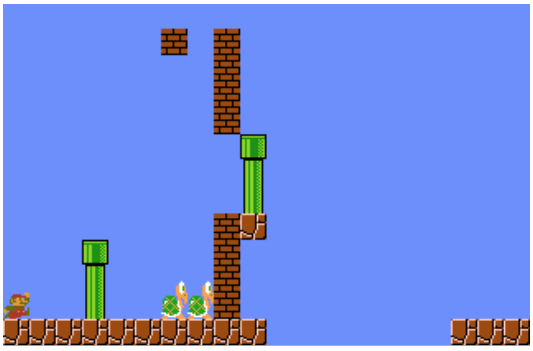
Figure 6.1 An example level that the MFF A* agent fails to complete in the Coin Punishing Model (where collecting a coin results in death) when `searchSteps` is set to 3. Lowering this parameter to 2 or 1 resolves the issue.

Consequently, for the Punishing Model scenarios, this configuration successfully generated levels where the specified mechanic was demonstrably required for the agent to find a solution, overcoming a key limitation noted with the Baumgarten's agent in the original study. Figure 6.2 provides an example comparing Shell Kill levels generated by the two agents, where the MFF A* version requires the shell interaction. Similarly, Figure 6.3 illustrates a Stomp Kill level comparison, highlighting how the MFF A* generated level enforces a mid-air stomp.

Because MFF A* actively searches for levels where the target mechanic is truly required, the resulting levels tend to be more complex. This increased complexity often leads to lower final fitness values compared to those generated by Baumgarten's agent. As a result, the average maximum fitness progression over generations tends to be similar to or even worse than that of Baumgarten's agent. This effect is observable, for instance, in the Stomp Punishing Model (see Figure 6.4 and Figure 6.3), where MFF A* sacrifices fitness score in favor of enforcing genuine mechanic activation.

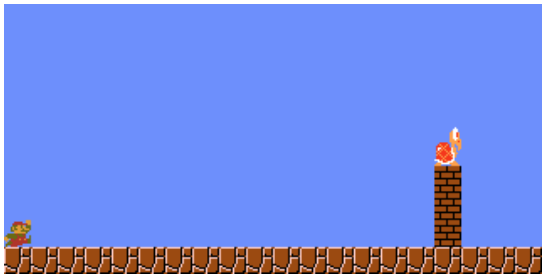


(a) Shell Kill level by Baumgarten's agent

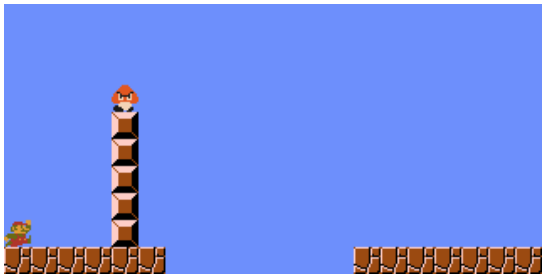


(b) Shell Kill level by the MFF A* agent

Figure 6.2 Comparison of Shell Kill levels generated by Baumgarten's agent (a) and the MFF A* agent (b). Compared to the Baumgarten's level, in the MFF A* level, the player is required to perform a shell kill to progress: first stomping a Koopa, then using its shell to destroy a wall blocking the path. During this sequence, the shell bounces after destroying the wall. The MFF A* level has worse fitness, but it fulfills the objective better. The Baumgarten's level was taken directly from the original paper.



(a) Stomp Kill level by Baumgarten's agent



(b) Stomp Kill level by the MFF A* agent



(c) Mario in motion, with a ghost trail illustrating his movement across the level

Figure 6.3 Comparison of Stomp Kill levels generated by Baumgarten's agent (a) and the MFF A* agent (b, c). Compared to the Baumgarten's level, in the MFF A* level, the player is required to perform a stomp kill mid-air to progress. The MFF A* level has worse level fitness, but it fulfills the objective better. The Baumgarten's level was taken directly from the original paper.

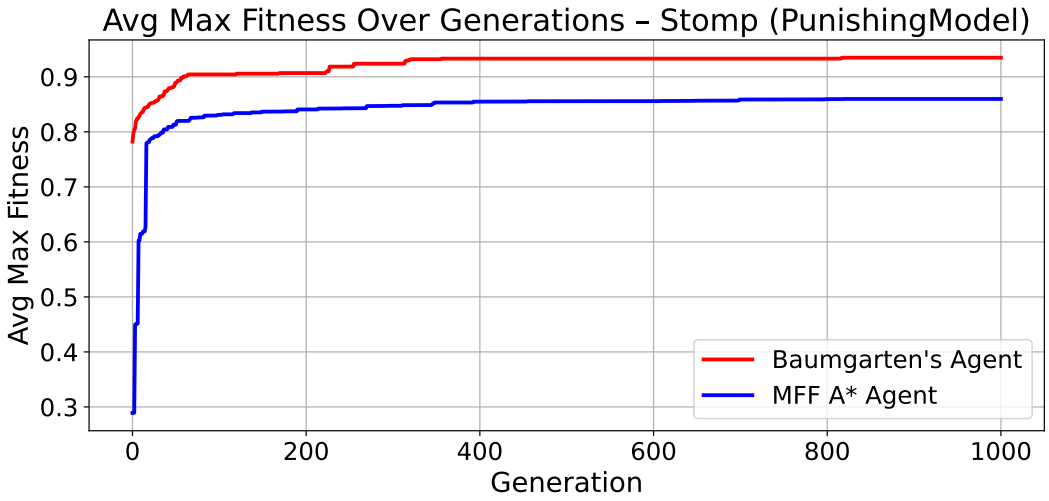


Figure 6.4 Average maximum fitness over generations for the Stomp Kill Punishing Model. The MFF A* agent (blue) searches for levels where the mechanic is truly required, resulting in more complex and harder levels. Consequently, the average fitness is similar or worse compared to Baumgarten's agent (red), which more easily finds high fitness values by bypassing the intended goal.

6.4.3 Combined Optimizations and Fitness Evolution

Implementing the full set of improvements, including multithreading, result caching, elitism fixes, and the use of MFF A* Plus, resulted in a dramatic performance increase, even with elite re-testing and `searchSteps` set to 1. The average run time for a 1000-generation experiment plummeted to approximately 25 minutes (0:25:01, see Table 6.1). This represents roughly 25-fold (2493%) speedup compared to the original average of 10-hour and 49-minute runtime with Baumgarten, making the MechanicsGen approach vastly more practical for experimentation and iteration. The caching, in particular, proved highly effective due to the frequent generation of duplicate scene structures during evolution. The non-determinism fixes also led to more stable and reliable evolutionary progress. Additionally, memory usage dropped significantly, from a peak of 54 GB RAM to just 2 GB (when utilizing 25 cores), making the system far more resource-efficient and easier to run on standard hardware.

Table 6.1 Comparison of agent runtimes for a 1000-generation MechanicsGen run. Average and median runtimes.

Agent Version	**Average Real Hours**	**Median Real Hours**
Baumgarten's Agent	10:48:33	11:15:36
MFF A*	3:30:38	3:34:13
MFF A* Plus	2:05:33	1:54:57
MFF A* Plus and Optimizations	0:25:01	0:19:13

Table 6.2 Comparison of agent runtimes for a 1000-generation MechanicsGen run. Maximum and minimum runtimes.

Agent Version	Max Real Hours	Min Real Hours
Baumgarten's Agent	16:57:25	5:16:03
MFF A*	5:18:30	2:08:39
MFF A* Plus	4:40:41	1:10:56
MFF A* Plus and Optimizations	1:34:54	0:03:35

Table 6.3 Resource usage comparison for MechanicsGen runs (1000 generations)

Agent Version / Setup	Average CPU Hours	Median CPU Hours	Max Used Memory
Baumgarten's Agent	183:05:10	212:48:09	54 GB
MFF A*	38:23:41	33:54:10	19 GB
MFF A* Plus	19:05:04	15:48:09	33 GB
MFF A* Plus and Optimizations	2:38:47	1:44:42	2 GB

6.5 Chapter Summary

MechanicsGen's goal to generate game scenes focused on specific mechanics using evolutionary algorithms guided by agent simulations. This chapter detailed the replication of the work, uncovering significant limitations in the original implementation, including incomplete code, faulty mechanisms, performance issues, and the inherent drawbacks of relying on the Baumgarten's A* agent for evaluation.

The core contribution of this thesis regarding MechanicsGen involved not only replicating, but substantially completing, correcting, and enhancing the framework. This included implementing missing components, fixing the evolutionary elitism, integrating the superior MFF A* agent (and an optimized Plus version), developing novel solutions to mitigate agent non-determinism during evaluation, and completely rearchitecting the execution model using multithreading and result caching.

The results demonstrated the clear performance benefits of MFF A* over Baumgarten's A*. More importantly, the combined technical improvements transformed the framework from a cumbersome, slow system into a highly efficient one, reducing average run times from hours to minutes. This work also highlighted the subtle relationship between agent search parameters and the generation of levels intended to enforce specific mechanics. Thanks to MFF A*, it became possible to generate clever level designs where the given mechanic had to be triggered to complete the level. While there are still some challenges in reliably ensuring that the mechanic is always necessary, the improved MechanicsGen framework with MFF A* provides a much more robust and practical platform for future research in intentional computational level design.

7 Additional Scripts and Documentation

This chapter provides brief overviews and pointers to the detailed documentation for the various frameworks and helper scripts used or developed as part of this work. The full documentation for each component can be found in the Attachments A.

7.1 TOAD-GAN

We utilized the original framework to generate sets of levels using pre-trained models (`original_1-1`, `original_3-3`) and a custom-trained model on `krys-1`. Minor modifications were made to the associated TOAD-GUI tool to facilitate batch generation and visualization for this thesis. The original project did not provide a built-in mechanism for running agents, so the Mario AI Framework was used separately for evaluating the playability of generated levels using different agents.

For comprehensive details on system requirements, setup instructions, training procedures, level generation, and our specific modifications, please refer to the TOAD-GAN Documentation in Attachments A.

7.2 DagstuhlGAN

This project integrates Java (for CMA-ES and simulation via the Mario AI Framework) and Python (for the PyTorch-based GAN). Significant effort was required to set up the environment due to its reliance on very old library versions, particularly PyTorch 0.3.1, which is incompatible with Windows. Key modifications involved updating the project to the newer version of the Mario AI Framework, adding new fitness functions, centralizing fitness evaluation logic, and modifying the level representation and visualization.

The detailed documentation covering core components, setup challenges and solutions (especially for macOS), and a list of implemented modifications is available in the DagstuhlGAN Documentation in Attachments A.

7.3 MechanicsGen

Our contributions to this project include a multithreaded implementation for significantly faster execution, the introduction of an evaluation caching system to avoid redundant simulations, and fixes to the elitism mechanism. We also added support for re-testing, made performance optimizations to both the MFF agent implementation (specifically the MFF A* Plus variant) and the Slim forward model, and extended the system with limited agent variants (for both MFF A* agent and Baumgarten's agent) with distinct behavioral constraints: an inability to run, a limited jump height, and an inability to perceive enemies, to allow targeted evaluation.

Further information on the implemented files, framework modifications, parameter configuration (`FI2PopParameters.txt`, `CMEParameters.txt`), and execution instructions for both legacy and threaded versions can be found in the MechanicsGen Documentation in Attachments A.

7.4 Helper Python Scripts

To support the analysis and data handling required for this project, several custom Python scripts were created. These scripts perform various tasks such as preprocessing level data into frames suitable for GAN training (DagstuhlGAN), calculating metrics from level representations, visualizing the evolution of fitness values over generations, comparing agent performance statistics, and extracting specific results (e.g., best-performing latent vectors).

A list describing the purpose of each key script is provided in the Helper Python Scripts section in Attachments A.

7.5 Metacentrum Job Scripts

To enable large-scale experimentation on the Metacentrum computational grid, a set of job submission scripts was developed. These scripts automate the compilation, setup, and parallel execution of both legacy and multithreaded versions of the MechanicsGen experiments using PBS. They support both agent configurations and streamline the process of distributing jobs with varied parameters.

Details on how each script functions, including submission structure and output handling, can be found in Attachments A.

8 Conclusion and Future Work

The goal of this thesis was to improve the evaluation phase within procedural content generation (PCG) for Super Mario Bros. levels, specifically addressing the limitations inherent in the widely adopted A* agent developed by Robin Baumgarten. The core hypothesis was that the known inefficiencies and limits of Baumgarten's agent may have significantly impacted the outcomes and interpretations of previous research that relied upon it for level validation or agent-in-the-loop generation. To investigate this, we focused on replacing Baumgarten's agent with the demonstrably superior MFF agents within the frameworks of three studies: TOAD-GAN, DagstuhlGAN, and MechanicsGen.

8.1 Summary of Key Findings

The replication, extension, and re-evaluation of the three selected studies yielded consistent results highlighting the significant advantages of employing the MFF agents:

1. **Improved Playability Assessment (TOAD-GAN):** Re-evaluating levels generated by TOAD-GAN using the MFF A* Grid agent revealed substantially higher completion rates compared to those reported using Baumgarten's agent. This confirmed that the original study underestimated the playability of its generated content due to the validator's limitations. Many levels previously deemed unplayable were, in fact, solvable by the more capable MFF A* agent. However, the analysis also underscored that inherent generator limitations in TOAD-GAN still produced uncompletable levels, independent of the evaluation agent.

2. **Enhanced Evolutionary Guidance (DagstuhlGAN):** Integrating MFF A* Grid into the Latent Variable Evolution (LVE) loop of DagstuhlGAN resulted in faster convergence toward desired fitness objectives (e.g., playable levels with many jumps). LVE itself became noticeably faster with the introduction of the MFF A* Grid. The MFF A* Grid agent navigated the latent space more effectively, achieving better fitness scores more rapidly and handling more complex base levels (like `krys-1`) far more efficiently than Baumgarten's agent, which struggled with computational cost. Furthermore, the MFF A* Grid agent avoided certain degenerate behaviors (like repeated jumping without progress) that the original agent sometimes exploited in the fitness function due to unintended artifacts of the agent. The exploration of new fitness functions, particularly the static grid-search jump analysis, demonstrated potential for rapid, albeit simplified, structural optimization.

3. **Enabling Intentional Design and Performance Boost (MechanicsGen):** The most dramatic impact was observed in the MechanicsGen framework. Replacing Baumgarten's agent with MFF A* (and its optimized MFF A* Plus variant), combined with architectural improvements (multithreading, caching) and bug fixes (elitism, non-determinism handling), resulted in a massive performance increase, reducing runtimes from over 10

hours to approximately 25 minutes and lowering memory usage from 54 GB to just 2 GB of RAM. More significantly, by carefully configuring the MFF A* agent (reducing `searchSteps`) and adding verification layers (Punishing Model Event Verification), it became possible to generate levels where the targeted game mechanic was genuinely required for completion, overcoming the original study's limitation where bypass routes were often found. This demonstrated that a more capable agent, when appropriately configured, can better fulfill the semantic goals of intentional level design.

Across all studies, the MFF A* agent proved faster, more reliable, and more complete than Baumgarten's agent. Its use provides a more accurate assessment of level playability and leads to more efficient and potentially more effective agent-in-the-loop generation processes.

8.2 Contributions

This thesis makes the following contributions:

- **Quantified Impact of Agent Choice:** Provided concrete, empirical evidence across multiple PCG frameworks demonstrating the significant impact of using a suboptimal versus a state-of-the-art evaluation agent in Super Mario level generation research.
- **Enhanced PCG Frameworks:** Successfully integrated the MFF A* and MFF A* Grid agent into the TOAD-GAN, DagstuhlGAN, and MechanicsGen systems, improving their evaluation accuracy and, for DagstuhlGAN and MechanicsGen, the efficiency and effectiveness of their generation processes.
- **Technical Rehabilitation and Extension:** Substantially improved the MechanicsGen framework by completing missing code, fixing critical bugs, addressing agent non-determinism, drastically optimizing performance, and modernizing its architecture.
- **Methodological Advancements:** Introduced and evaluated novel fitness functions within DagstuhlGAN (static analysis) and refined agent configurations and verification methods within MechanicsGen to better achieve intentional design goals.
- **Improved Documentation and Reproducibility:** Documented the often complex setup procedures and technical challenges encountered during replication, providing valuable resources for future researchers wishing to build upon these works.

8.3 Limitations and Future Work

Despite the positive results, this work has limitations that open avenues for future research:

- **Generator Quality:** While this thesis improved the evaluation of generated levels, it did not fundamentally alter the underlying level generators (TOAD-GAN, DagstuhlGAN's DCGAN). The quality and complexity limitations of these generators, particularly the small, fragmented levels from DagstuhlGAN, remain. Future work should focus on improving the generators themselves, now equipped with more reliable validation tools.

- **Scope of Re-evaluation:** Only three studies were re-evaluated. Many other PCG papers have used Baumgarten's agent; extending this comparative analysis to a broader range of studies could provide further insights.

- **Addressing Non-Determinism:** Although the agents are intended to be deterministic, inconsistencies in execution caused by factors such as JVM thread scheduling and flawed time synchronization can lead to different outcomes on the same level (see section 2.5.2). Future work could investigate more robust mechanisms to ensure reproducible behavior across evaluations.

- **Path Deviation Handling:** During the evaluation of TOAD-GAN generated levels, we observed that in rare cases, the A* grid search fails to find a valid path to the goal—typically due to large or high obstacles. However, the MFF A* Grid agent was sometimes still able to complete the level by exploiting environmental features, mostly nearby enemies (e.g., bouncing off them). Since the precomputed path ended at the obstacle, the agent's deviation from the path triggered an internal error and we had to classify it as a failure. Future work could improve path-tracking to detect valid off-path completions and avoid false failure reports.

- **Beyond Playability:** The focus was primarily on level completability. Future work could explore objectives such as maximizing coin collection, defeating all enemies, or collecting powerups. These goals are currently ignored by the implemented agents.

- **Agent Capabilities:** While the MFF A * grid is highly capable, it is still not flawless (e.g., requiring manual adjustment of parameters such as `jumpBoost` or `searchSteps` to finish a level). Exploring its limits on even more complex, human-designed, or adversarially generated levels could be informative.

In conclusion, this thesis successfully demonstrated that replacing the historically standard but limited Baumgarten A* agent with the modern, efficient MFF agents significantly improves the evaluation and generation of Super Mario Bros. levels. The MFF agents yield more accurate playability assessments, enable faster and more reliable evolutionary search, and offer better control over design intent. This work corrects potential misinterpretations in earlier studies and provides a stronger foundation, along with tools and documentation, for future advances in procedural content generation. These technical improvements also offer practical resources that support further research.

Bibliography

1. Alkaim, Artur. Procedural Generation for Architecture. *Computer Science*. 2015.
2. Umenhoffer, Tamás; Magdics, Milán; Zsolnai, Károly. Procedural generation of hand-drawn like line art. *Képfeldolgozók és Alakfelismerok VIII. Konferenciája*. [N.d.], pp. 502–514.
3. Smith, Gillian. An Analog History of Procedural Content Generation. In: *FDG*. Boston, MA, 2015, pp. 1–6.
4. Nintendo Co., Ltd. *Mario History* [`https://mario.nintendo.com/history/`]. n.d. Accessed: 2025-04-30.
5. Togelius, Julian; Karakovskiy, Sergey; Baumgarten, Robin. The 2009 mario ai competition. In: *IEEE Congress on Evolutionary Computation*. IEEE, 2010, pp. 1–8.
6. Šosvald, David; Töpfer, Michal; Holan, Jan; Černý, Vojtěch; Gemrot, Jakub. Super Mario A-Star Agent Revisited. In: *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 2021, pp. 1008–1012.
7. Awiszus, Maren; Schubert, Frederik; Rosenhahn, Bodo. TOAD-GAN: Coherent style level generation from a single example. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*. 2020, vol. 16, pp. 10–16. No. 1.
8. Volz, Vanessa; Schrum, Jacob; Liu, Jialin; Lucas, Simon M; Smith, Adam; Risi, Sebastian. Evolving mario levels in the latent space of a deep convolutional generative adversarial network. In: *Proceedings of the genetic and evolutionary computation conference*. 2018, pp. 221–228.
9. Khalifa, Ahmed; Green, Michael Cerny; Barros, Gabriella; Togelius, Julian. Intentional computational level design. In: *Proceedings of The Genetic and Evolutionary Computation Conference*. 2019, pp. 796–803.
10. Šosvald, David. Optimizing Super Mario game tree search [`https://dspace.cuni.cz/handle/20.500.11956/184179`]. 2023.
11. Fandom contributors. *World 1-1 - Gameplay - Super Mario Bros. [Screenshot]*. 2025. Available also from: `https://nintendo.fandom.com/wiki/World_1-1_(Super_Mario_Bros.)?file=World_1-1_-_Gameplay_-_Super_Mario_Bros.png`. Accessed: 2025-05-05.
12. Fandom contributors. *World 1-2 - Gameplay - Super Mario Bros. [Screenshot]*. 2025. Available also from: `https://nintendo.fandom.com/wiki/World_1-2_(Super_Mario_Bros.)?file=World_1-2_-_Gameplay_-_Super_Mario_Bros.png`. Accessed: 2025-05-05.
13. Khalifa, Ahmed. *Mario AI Framework repository* [`https://github.com/amidos2006/Mario-AI-Framework`]. 2019. Accessed: 2025-04-30.
14. Summerville, Adam James; Snodgrass, Sam; Mateas, Michael; Ontanón, Santiago. The vglc: The video game level corpus. *arXiv preprint arXiv:1606.07487*. 2016.

15. Togelius, Julian; Shaker, Noor; Karakovskiy, Sergey; Yannakakis, Georgios N. The mario ai championship 2009-2012. *Ai magazine*. 2013, vol. 34, no. 3, pp. 89–92.

16. Šosvald, David. *Super Mario A-Star Agents* [`https://gitlab.com/gamedev-cuni-cz/theses/super-mario-astar`]. 2023. Repository of the Bachelor and Master thesis at the Faculty of Mathematics and Physics, Charles University. Accessed: 2025-04-30.

17. Lucas, Simon M; Volz, Vanessa. Tile pattern KL-divergence for analysing and evolving game levels. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. 2019, pp. 170–178.

18. Ashlock, Daniel. Automatic generation of game elements via evolution. In: *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games*. IEEE, 2010, pp. 289–296.

19. Green, Michael Cerny; Khalifa, Ahmed; Barros, Gabriella AB; Nealen, Andy; Togelius, Julian. Generating levels that teach mechanics. In: *Proceedings of the 13th International Conference on the Foundations of Digital Games*. 2018, pp. 1–8.

20. Goodfellow, Ian J; Pouget-Abadie, Jean; Mirza, Mehdi; Xu, Bing; Warde-Farley, David; Ozair, Sherjil; Courville, Aaron ; Bengio, Yoshua. Generative adversarial nets. *Advances in neural information processing systems*. 2014, vol. 27.

21. Radford, Alec; Metz, Luke; Chintala, Soumith. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*. 2015.

22. Shaham, Tamar Rott; Dekel, Tali; Michaeli, Tomer. Singan: Learning a generative model from a single natural image. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 4570–4580.

23. Gulrajani, Ishaan; Ahmed, Faruk; Arjovsky, Martin; Dumoulin, Vincent; Courville, Aaron C. Improved training of wasserstein gans. *Advances in neural information processing systems*. 2017, vol. 30.

24. Arjovsky, Martin; Chintala, Soumith; Bottou, Léon. Wasserstein generative adversarial networks. In: *International conference on machine learning*. PMLR, 2017, pp. 214–223.

25. Vikhar, Pradnya A. Evolutionary algorithms: A critical review and its future prospects. In: *2016 International Conference on Global Trends in Signal Processing, Information Computing and Communication (ICGTSPICC)*. 2016, pp. 261–265. Available from doi: 10.1109/ICGTSPICC.2016.7955308.

26. Kimbrough, Steven Orla; Koehler, Gary J; Lu, Ming; Wood, David Harlan. On a feasible–infeasible two-population (fi-2pop) genetic algorithm for constrained optimization: Distance tracing and no free lunch. *European journal of operational research*. 2008, vol. 190, no. 2, pp. 310–327.

27. Hansen, Nikolaus; Müller, Sibylle D; Koumoutsakos, Petros. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary computation*. 2003, vol. 11, no. 1, pp. 1–18.

28. Khalifa, Ahmed; Lee, Scott; Nealen, Andy; Togelius, Julian. Talakat: Bullet hell generation through constrained map-elites. In: *Proceedings of The Genetic and Evolutionary Computation Conference*. 2018, pp. 1047–1054.

29. Snodgrass, Sam; Ontañón, Santiago. Experiments in map generation using Markov chains. In: *FDG*. 2014.

30. Snodgrass, Sam; Ontañón, Santiago. A hierarchical approach to generating maps using markov chains. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*. 2014, vol. 10, pp. 59–65. No. 1.

31. Russell, Stuart; Norvig, Peter. *Artificial Intelligence: A Modern Approach*. 3rd. Prentice Hall, 2010. isbn 978-0-13-604259-4.

32. Togelius, Julian; Karakovskiy, Sergey; Koutník, Jan; Schmidhuber, Jurgen. Super mario evolution. In: *2009 ieee symposium on computational intelligence and games*. IEEE, 2009, pp. 156–161.

33. Green, Michael Cerny; Mugrai, Luvneesh; Khalifa, Ahmed; Togelius, Julian. Mario level generation from mechanics using scene stitching. In: *2020 IEEE Conference on Games (CoG)*. IEEE, 2020, pp. 49–56.

34. Kerssemakers, Manuel; Tuxen, Jeppe; Togelius, Julian; Yannakakis, Georgios N. A procedural procedural level generator generator. In: *2012 IEEE Conference on Computational Intelligence and Games (CIG)*. 2012, pp. 335–341. Available from doi: 10.1109/CIG.2012.6374174.

35. Hauck, Eduardo; Aranha, Claus. Automatic Generation of Super Mario Levels via Graph Grammars. In: *2020 IEEE Conference on Games (CoG)*. 2020, pp. 297–304. Available from doi: 10.1109/CoG47356.2020.9231526.

36. Snodgrass, Sam; Ontanon, Santiago. A hierarchical mdmc approach to 2d video game map generation. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*. 2015, vol. 11, pp. 205–211. No. 1.

List of Figures

2.1 Gameplay screenshots taken from the levels World 1-1 and World 1-2 in Super Mario Bros. [11, 12]. 12

4.1 Visualization of the Multi-Scale architecture of TOAD-GAN. (Image taken from the original paper and slightly resized to fit the page layout) [7]. 25
4.2 Comparison between the original floating platform level 3-3 (a) and a level generated (b) by a TOAD-GAN model trained on it. While the original Baumgarten A* agent failed to complete the generated level, the improved MFF A* Grid agent successfully reached the end. The generated level convincingly captures stylistic elements like recurring platform layouts and vertical structure. 27
4.3 The obstacle extracted from the level that Baumgarten's agent could finish and MFF A* Grid could not due to a higher search steps parameter value. 28

5.1 The DCGAN architecture used in DagstuhlGAN, showing the generator, discriminator, and the training process. Image taken from the original paper [8]. 31
5.2 Latent vector optimization using CMA-ES and a fixed generator. Image from the original paper [8]. 32
5.3 Visualization of the changes done to the levels done for compatibility with the new Framework. 34
5.4 Average fitness progression (lower is better) over 1000 CMA-ES iterations using the original jump fitness (F_1) on the generator trained on the Original level 1-1. MFF A* Grid converges faster to better fitness values. 37
5.5 Average fitness progression using the original jump fitness (F_1) on the generator trained on `krys-2`. MFF A* Grid converges faster, but final fitness is similar. 38
5.6 Example levels found during optimization with the original jump fitness (F_1). (a) Level generated using Baumgarten's Agent. (b) Level generated using the MFF A* Grid agent. Both levels correspond to some of the lowest fitness values found. 38
5.7 Fitness progression of the Run-Time test. Lower values indicate slower level completion. This metric serves as a proxy for testing agents—longer runtimes may suggest weaknesses in pathfinding. Baumgarten's agent is already slow from the first generation and continues to perform poorly. In contrast, MFF A* maintains a stable, low runtime, reflecting higher efficiency, though it likely struggles to maximize runtime due to stronger pathfinding. MFF A* Grid gradually found levels that took a bit longer to complete. 39

5.8 Example levels with some of the lowest fitness values found during the runtime-based optimization. (a) Level generated by Baumgarten's Agent, which appears relatively straightforward and quick to complete. (b) Level generated by MFF A*, which includes more obstacles or pathing complexity, requiring more time to complete. (c) Level generated by MFF A* Grid. 40
5.9 Average fitness progression using Backtracked Nodes fitness (F_{nodes}) on the generator trained on the Original level 1-1. Optimization aims for solvable levels with high backtracking (lower scores). Baumgarten's agent reached lower values and hits a limit due to inefficiency on harder levels. MFF A* and MFF A* Grid effectively optimize by finding and solving increasingly challenging levels that induce more backtracking. 41
5.10 Average fitness progression (lower is better) over 1000 CMA-ES iterations using the Grid-Search Jump Fitness (F_{static_jumps}). Both plots compare the standard heuristic (`Static Analyzer`) including single-tile gaps with a variant excluding them (`Static Analyzer Without One Gaps`). Optimization using static analysis is effective and rapid for both the generator trained on (a) Original level 1-1 and (b) the `krys-2` level. 42
5.11 Example levels with some of the lowest fitness values found using the Grid-Search Jump Fitness. (a) Level evolved by the static analyzer excluding single-tile gaps, trained on the Original level 1-1. While aesthetically pleasing, the level exploits the heuristic by generating a two-tile-wide stair structure that artificially increases the jump count. (b) Level evolved using the standard heuristic on the `krys-2` generator, which takes advantage of one-tile-wide gaps to maximize jump potential. (c) Level evolved with the one-gap heuristic disabled, resulting in a structure where many of the jumps are genuinely required for traversal, illustrating a more meaningful expression of platforming complexity. 43

6.1 An example level that the MFF A* agent fails to complete in the Coin Punishing Model (where collecting a coin results in death) when `searchSteps` is set to 3. Lowering this parameter to 2 or 1 resolves the issue. 51
6.2 Comparison of Shell Kill levels generated by Baumgarten's agent (a) and the MFF A* agent (b). Compared to the Baumgarten's level, in the MFF A* level, the player is required to perform a shell kill to progress: first stomping a Koopa, then using its shell to destroy a wall blocking the path. During this sequence, the shell bounces after destroying the wall. The MFF A* level has worse fitness, but it fulfills the objective better. The Baumgarten's level was taken directly from the original paper. 52

6.3 Comparison of Stomp Kill levels generated by Baumgarten's agent (a) and the MFF A* agent (b, c). Compared to the Baumgarten's level, in the MFF A* level, the player is required to perform a stomp kill mid-air to progress. The MFF A* level has worse level fitness, but it fulfills the objective better. The Baumgarten's level was taken directly from the original paper. 52
6.4 Average maximum fitness over generations for the Stomp Kill Punishing Model. The MFF A* agent (blue) searches for levels where the mechanic is truly required, resulting in more complex and harder levels. Consequently, the average fitness is similar or worse compared to Baumgarten's agent (red), which more easily finds high fitness values by bypassing the intended goal. 53

List of Tables

4.1 Performance of Baumgarten's Agent and MFF A* Grid on various level generators 28

6.1 Comparison of agent runtimes for a 1000-generation MechanicsGen run. Average and median runtimes. 53
6.2 Comparison of agent runtimes for a 1000-generation MechanicsGen run. Maximum and minimum runtimes. 54
6.3 Resource usage comparison for MechanicsGen runs (1000 generations) 54

List of Abbreviations

A* A-star (Informed Search Pathfinding Algorithm)

AI Artificial Intelligence

BND Best Nodes evaluated (Parameter configuration for MFF A* Grid Agent)

CMA-ES Covariance Matrix Adaptation Evolution Strategy

CMElites Constrained MAP-Elites (Quality Diversity Algorithm)

CPU Central Processing Unit

D Discriminator (Component of a GAN)

DagstuhlGAN *Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network* (Short name used for reference [8])

DCGAN Deep Convolutional Generative Adversarial Network

EA Evolutionary Algorithm

FI2Pop Feasible-Infeasible 2-Population (Evolutionary Algorithm)

G Generator (Component of a GAN)

GAN Generative Adversarial Network

GB Gigabyte

Grid Search Static pathfinding pre-processing step used by MFF A* Grid agent

I/O Input/Output

JDK Java Development Kit

JVM Java Virtual Machine

LVE Latent Variable Evolution

MechanicsGen *Intentional Computational Level Design* (Short name used for reference [9])

MFF A* Improved A* agent from Šosvald et al.

MFF A* Grid Variant of MFF A* using grid pre-processing

MFF A* Plus Optimized variant of MFF A* and Slim forward model, incorporating further optimizations to the Slim Forward Model

MFF Agents Collective term for the MFF A*, MFF A* Grid, and MFF A* Plus agents.

NES Nintendo Entertainment System

PBS Portable Batch System

PCG Procedural Content Generation

RAM Random Access Memory

ReLU Rectified Linear Unit (Activation Function)

RGB Red, Green, Blue (Color Model)

SDK Software Development Kit

SinGAN Single Image Generative Adversarial Network

Slim Forward Model Optimized forward model for Mario AI Framework from Šosvald et al.

SPECfp2017 Standard Performance Evaluation Corporation floating-point benchmark 2017

TOAD-GAN Token-based One-shot Arbitrary Dimension Generative Adversarial Network (Short name used for reference [7])

TPKL-Div Tile Pattern Kullback-Leibler Divergence (Evaluation Metric)

VGLC Video Game Level Corpus

WGAN Wasserstein Generative Adversarial Network

WGAN-GP Wasserstein Generative Adversarial Network with Gradient Penalty

A Attachments

TOAD-GAN Documentation

System and Environment

- Operating System: Windows 11 (although Linux and macOS should work as well)
- Programming Language: Python 3
- Libraries: `pytorch`, `pillow`, `numpy`, `wandb`, `torchvision`, `tqdm`, `umap-learn`, `loguru`, `matplotlib`, `seaborn`, `PyYAML`, `scikit-learn`, `py4j`
- Development Environment: Visual Studio Code

Implemented Modifications

- The function `save_txt` in `GUI.py` (TOAD_GUI) was modified to allow generating and saving 100 levels at once.
- A pre-trained model on the level `krys-1` was added to the `generators/v2` folder.
- The `levels` folder includes generated levels based on the models trained on `original_1`, `original_3-3`, and `krys-1`. It also includes a file `random-orig-levels` containing 1000 levels generated using randomly selected models based on original levels.
- TOAD-GAN itself was not modified, only used for training new models; thus, it is not part of the repository.

Installation

First clone the repository

```
git clone https://github.com/Mawiszus/TOAD-GUI
```

To install and set up TOAD-GAN, follow the instructions in its README file. First, install all dependencies using:

```
pip3 install -r requirements.txt -f
↪  "https://download.pytorch.org/whl/torch_stable.html"
```

Training a Model

Once dependencies are installed, TOAD-GAN can be trained by running `main.py`. To train a model on Super Mario Bros. level 1-1 with 4000 iterations per level, run:

```
python main.py --input-dir input --input-name
↪  lvl_1-1.txt --num_layer 3 --alpha 100 --niter 4000
↪  --nfc 64
```

Generating Levels

To generate levels using a pre-trained TOAD-GAN model, run:

```
python generate_samples.py --out_path path/to/generator
↪  --input-dir input --input-name lvl_1-1.txt
↪  --num_layer 3 --alpha 100 --niter 4000 --nfc 64
```

We created three different TOAD-GAN models based on three different levels: `original_1-1`, `original_3-3`, and `krys-1` (a highly complex level). Using these models, we generated 100 unique levels. These models and the generated levels are included in the repository.

TOAD-GUI Extension

The original author developed an extension called TOAD-GUI for visualizing and manipulating generated levels. This framework improves interaction with generated levels and simplifies editing. We made a small modification to the source code to enable batch generation of levels for a trained model. We also utilized it's visualization functionality to add a button that generates level images specifically for this thesis.

Level Testing

TOAD-GAN does not include a built-in tool to play levels with an AI agent. For testing, we used the Mario AI Framework available on GitLab, which includes both the MFF A* agent and Baumgarten's agent. We evaluated the generated levels using these agents within the framework.

DagstuhlGAN Documentation

System and Environment

- Operating System: macOS (Linux may work; Windows is not recommended)
- Programming Languages: Java, Python 3.6
- Libraries: PyTorch 0.3.1, torchvision 0.2.0
- Java SDK: Amazon Corretto 17

Core Components

- `DagstuhlGAN-master/marioaiDagstuhl/src/cmates/CMAMarioSolver`
  - Main entry point for CMA-ES evolution.
  - Controls number of generations (iterations) and number of levels.
  - Generates output files in the format `timeline#.txt`, each representing a generation of vectors with corresponding fitness values.
  - Fitness values should generally decrease over time.
- `./marioaiDagstuhl/src/cmates/MarioEvalFunction`
  - Contains the `valueOf` function, which serves as a dispatcher for evaluating latent vectors.
  - Supports multiple fitness functions such as `levelGridJumpFitness`, `levelBacktrackFitness`, etc.
  - Includes utilities for converting latent vectors to levels.
- `./marioaiDagstuhl/src/communication/MarioProcess`
  - Responsible for level simulation and fitness evaluation.
  - Includes agent selection (must be manually chosen which agent to use).
  - Handles string-based level representation required by newer versions of the Mario AI Framework.
- `./marioaiDagstuhl/src/viewer`
  - `MarioLevelPlayer`: Play levels manually. (Input vector as an argument)
  - `MarioLevelSaver`: Save levels to file.(Input vector as an argument)
  - `MarioLevelViewer`: Render visual preview of a level. (Input vector as an argument)
  - `marioLevelBatchViewer.java`:
    - Tool for batch generation of level text files and corresponding images.

        * Input: `imagesVector.txt` containing one latent vector per line.
        * Output: A folder with .txt level file and level images for each vector.

- `DagstuhlGAN-master/pytorch`
  - GAN model training and generation in Python.
  - `gan_optimize.py` allows model selection logic modification.
  - `generator_ws.py` loads different pretrained models.
  - **Custom Generators**:
    * The custom trained `krys-2` generator is stored in the file `netG_epoch_14900_0_32.pth`.
    * To use this generator instead of the default `netG_epoch_5000.pth` (used for the original 1-1 level generation), all references to the original model must be replaced by the custom file throughout the project.

Implemented Modifications

- Fitness computation logic was centralized in `valueOf`.
- Support for new string-based level representation added.
- Agent modifications:
  - Baumgarten's agent tracks backtracked nodes.
  - MFF agent tracks jump counts.
- `simulateOneLevel` updated to support new fitness metrics.
- New level saving and visualization tools added under `viewer`.
- Java class files in `marioobjectives` appear unused and likely legacy.

Setup Instructions

1. IntelliJ Setup

- Open the root `DagstuhlGAN` project directory in IntelliJ.
- Go to `File → Project Structure...`:
  - Language Level: `SDK default`
  - Compiler Output: Set a custom output path
- Add module:
  - Go to `Modules → + → Import Module` and select `marioaiDagstuhl`.
  - Set Module SDK to `Amazon Corretto 17`.

- After adding the module, check that the required libraries are included and correctly linked in the module settings.

- Add Python 3.6 module for the `pytorch` folder.
- Set run/debug configuration to Java 17 (Corretto 17).
- Create file `my_python_path.txt` with the Python path, e.g.:

```
/Library/Frameworks/Python.framework/Versions/3.6
/bin/python3.6
```

- **Note:** If using the original version of the project without code modifications, Amazon Corretto 1.8.0 (JDK 8) must be used instead of Corretto 17.

2. PyTorch Installation Issues

- PyTorch 0.3.1 is not compatible with Windows.
- Cannot be installed via pip:

```
pip install torch==0.3.1   # Does not work
```

- Use macOS or Linux with Python 3.6 and install from a wheel (.whl) file.
- Required versions:
  - `torch==0.3.1`
  - `torchvision==0.2.0`

3. macOS Installation (Tested)

- System: macOS Monterey 12.7.4 (MacMini 7.1)
- Steps:
  1. Install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.
githubusercontent.com/Homebrew/install/HEAD/install.
sh)"
```

  2. Install pyenv:

```
brew install pyenv
```

3. Install Python 3.6.15:

```
pyenv install 3.6.15
pyenv global 3.6.15
```

4. Download PyTorch 0.3.1 .whl file from PyTorch archive
   `https://pytorch.org/get-started/previous-versions/`
5. Install PyTorch 'manually':

```
python3.6 -m pip install /path/to/torch-0.3.1-cp36-
cp36m-macosx_10_7_x86_64.whl
```

6. Install torchivision 0.2.0:

```
python3.6 -m pip install torchvision==0.2.0
```

MechanicsGen Documentation

Overview of Implemented Changes

- **FI2PopThreadChildRunnerMFF.java**: Implements the logic executed by each thread during parallel execution. Each thread independently evaluates a subset of the population using the selected experiment type.
- **FI2PopThreadParentRunnerMFF.java**: Controls and synchronizes all child threads. It handles population initialization, child thread input distribution, possible re-tests, result aggregation, and manages the evolutionary process across all iterations.
- **FI2PopChildRunner.java**: Original child runner extended with the missing agent implementations and new event types, such as `RUN` and `HIGH_JUMP`.
- **FI2PopChildRunnerMFF.java**: A variant of `FI2PopChildRunner.java` using MFF agents.
- **Added Agent Variants**:
  - `limitedEnemyBaumgarten`: The Baumgarten's agent variant 'blind' to enemies.
  - `limitedJumpBaumgarten`: The Baumgarten's agent variant with limited jump
  - `limitedRunBaumgarten`: The Baumgarten's agent variant which can't run..
  - `limitedEnemyAstar`: The MFF A* agent variant "blind" to enemies.
  - `limitedJumpAstar`: The MFF A* agent variant with limited jump
  - `limitedRunAstar`: The MFF A* agent variant which can't run.
- **Chromosome.java**: Enhanced to support execution of MFF agents and to integrate with result caching to prevent redundant simulations. We also fixed the calculation of the secondary fitness function to the same form described in the original paper. This file serves as a representation of the level, handles agent execution, and computes and stores evaluation results.
- **EvaluationCache.java** & **CachedResult.java**: Introduce a caching system to store previously evaluated level results. This significantly reduces redundant computations across generations, especially when dealing with multiple agents.
- **AgentBenchmarkGame.java**: Updated to record additional in-game statistics, such as number of kills, collected coins, and power-ups etc.

- **CMEThreadParentRunnerMFF.java** & **CMEThreadChildRunnerMFF.java**: Thread-based versions for the CME algorithm, enabling efficient parallel execution similar to FI2Pop.

  Note: While not significantly modified, two core components are crucial to the system's functioning:

  - `FI2PopGeneticAlgorithm.java`: Contains the main logic of the FI2Pop algorithm and does the evolutionary process.
  - `SlicesLibrary`: Stores all the different level slices (two versions: `Unique` and `Repeated`).

Framework Modifications

We reintroduced support for event triggers and extended the agent-game interface. The Slim forwarding model now supports `kill` events and new `must-do` event functionality. Furthermore, to reduce overhead, the agents now access the world using the `getCurWorld()` method instead of cloning the world state. Some small optimizations were also made to the Slim forwarding model to improve runtime performance.

Usage and Execution

MechanicsGen was developed using **Java 17**.

Parameters

Parameters for the evolutionary process are read from `FI2PopParameters.txt` and `CMEParameters.txt`, depending on the algorithm. Below is an example configuration of the FI2PopParameters file:

```
seed = 1023122
inputFolder = input_child/
outputFolder = output_child/
levelFolder = levels/
resultFolder = result/
agentType = coin
experimentType = punishingmodel
slicesType = Repeating
appendingSize = 3
chromosomeLength = 14
populationSize = 100
crossover = 0.7
mutation = 0.3
elitism = 1
fitnessType = entropy
```

Available values for `agentType` are: `Run, Coin, Mushroom, HighJump, Shell, Stomp, LimitedJump, EnemyBlind, NoRun`.
Available values for `experimentType` are: `PunishingModel, LimitedAgent`.

Legacy Execution (Non-threaded)

The original FI2Pop setup uses a parent-child process layout:

- Launch multiple instances (e.g., 25) of `FI2PopChildRunner` or `FI2PopChildRunnerMFF`, each with a unique `id` and the number of levels to process as arguments.
- Then, launch `FI2PopParentRunner` with the number of desired iterations as an argument.

Each child processes levels and writes outputs to the directory specified by `outputFolder`. The parent collects these outputs, computes fitness, and stores intermediate and final results in the `resultFolder`. The number of levels per child is determined by:

$$\texttt{levelsPerChild} = \frac{\texttt{populationSize}}{\texttt{numberOfChildren}}$$

The result folder includes:

- `results.txt`: Fitness statistics and iteration summaries.
- One folder per finished iteration: Contains level files, genes, and associated fitness values.

Threaded Execution (Recommended)

The new multithreaded versions simplify execution and eliminate intermediate I/O:

- Only run `FI2PopThreadParentRunnerMFF` with arguments `<maxIterations>` and `<numberOfThreads>`.
- Example:

```
javac -cp src src/FI2PopThreadParentRunnerMFF.java
java -cp src FI2PopThreadParentRunnerMFF 1000 25
```

- All configuration remains in `FI2PopParameters.txt`, which is fully compatible with both thread and non-thread versions.
- Child threads directly share memory with the parent. No intermediate files are created—only final results are stored in the `resultFolder`.

To reduce disk I/O and benefit from the gradually improving speed of the cache, results are saved only after the 50th iteration and then every `maxIterations / 10` (e.g., every 100 iterations in a 1000-iteration run).

Helper Python Scripts

Several Python scripts have been created for this project, focused on analyzing and working with data from the studied scientific papers. These scripts facilitate tasks such as data preprocessing, result visualization, etc. The scripts utilize the `matplotlib` and `numpy` libraries, and Python 3.12.3 was used for development.

marioLevelJumpAnalyzer.py

This script calculates the required number of jumps from the string representation of the "grid" path found by the GridSearch algorithm. It was later rewritten in Java for use in computing the fitness function based on grid jumps. (Used for Grid-search fitness)

marioDagstuhlFrames.py

In the original project of DagstuhlGAN, there were no necessary scripts for data preprocessing for model training. This script generates individual "frames" from levels, which serve as training data for the GAN.

marioDagstuhlGraph.py

This script is used for data visualization of the data from DagstuhlGAN. It generates graphs for analyzing the evolution of fitness values and other metrics from directories containing timeline files.

marioMechanicsGraph.py

This script visualizes the average max fitness over generations for different agent strategies in MechanicsGen experiments. It parses multiple `result.txt` files, averages results across 5 runs, and plots comparisons between the Baumgarten's and MFF A* agents

marioDagstuhlGetLowest.py

This script searches for the lowest fitness value from a given directory containing timeline files and returns the corresponding latent vector that achieved that value. Created for the DagstuhlGAN project.

marioCalculateResult.py

This script is used to calculate the success of agents based on the results contained in a given file, comparing the performance of two agents (Baumgarten's agent vs. MFF A* Grid). (Originally used for working with levels generated by TOAD-GAN, but it is universal and can be applied to other levels as well.)

Metacentrum Scripts

Several bash scripts have been created for running the MechanicsGen project in the Metacentrum grid. These scripts automate large-scale experimental runs, parameterized by agent type, evolutionary strategy, and repetition. Each script handles job submission using PBS (Portable Batch System) and ensures results are stored in the proper directory hierarchy.

metacentrum_allTests.sh

This script runs all the limited agents test and punishing model tests across all strategies. For each configuration (1–5 runs), it submits a separate job using the `metacentrum_FI2Pop_Baumgarten.sh` (default) and `metacentrum_FI2Pop_MFF.sh` PBS scripts. The `PARAM_FILE` and `FINAL_RESULT_DIR` variables are passed to each job.

metacentrum_allTests_threads.sh

Similar to `metacentrum_allTests.sh`, but targets the threaded version of the experiments. It uses `metacentrum_FI2Pop_threads_MFF.sh` to run experiments in parallel, submitting each configuration as a single job.

metacentrum_FI2Pop_Baumgarten.sh

A PBS job script that runs a single experiment using the Baumgarten's agent. It sets up scratch space, copies the necessary files (codebase, parameter file, and levels), compiles the required Java classes, and executes 25 instances of `FI2PopChildRunner` in parallel, each with a unique thread ID, followed by a `FI2PopParentRunner`. Final results are saved from scratch to the designated `FINAL_RESULT_DIR`.

metacentrum_FI2Pop_MFF.sh

A PBS job script that runs a single experiment using the MFF agent. It follows the same structure: sets up scratch space, copies required files, compiles the Java code, and launches 25 parallel instances of `FI2PopChildRunnerMFF`, each with a unique thread ID, followed by a `FI2PopParentRunner`. Final results are saved from scratch to the designated `FINAL_RESULT_DIR`.

metacentrum_FI2Pop_threads_MFF.sh

This PBS script launches the threaded variant of the MFF-based FI2Pop experiment. It uses `FI2PopThreadParentRunnerMFF` to handle both the evolutionary loop and evaluation internally using multiple threads.